

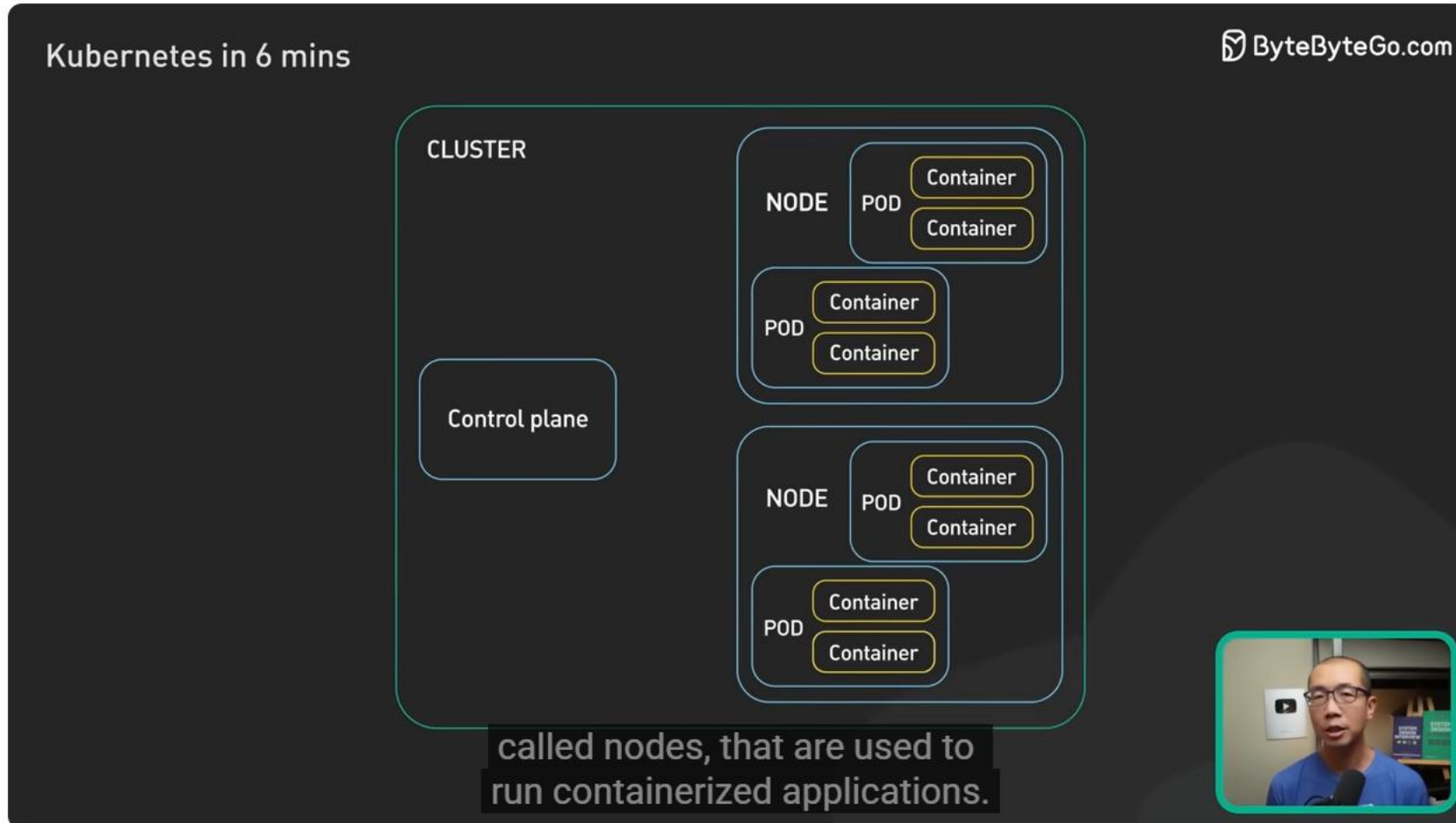
Kubernetes

Table des matières

Overview	2
Kubernetes Explained in 6 Minutes k8s Architecture	2
Comment apprendre Kubernetes en 2025	7
Cloud Native vs Cloud First vs Cloud Ready vs Born in the Cloud: The Ultimate Showdown	12
Migration	13
Migrating to Kubernetes + Best Practices for Cloud Native • T. Vitale & L. Højgaard • GOTO 2021	13
Migrating to Kubernetes	14
Learning	14
Kubernetes Tutorial for Beginners [FULL COURSE in 4 Hours]	14
Installer et configurer kubectl Kubernetes	84

Overview

[Kubernetes Explained in 6 Minutes | k8s Architecture](#)



Kubernetes Explained in 6 Minutes | k8s Architecture

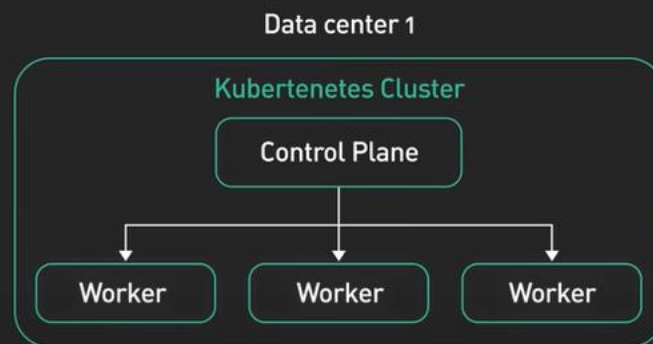


The first is the control plane.



Kubernetes in 6 mins

ByteByteGo.com



In production environments,
the control plane usually

Lire (k)



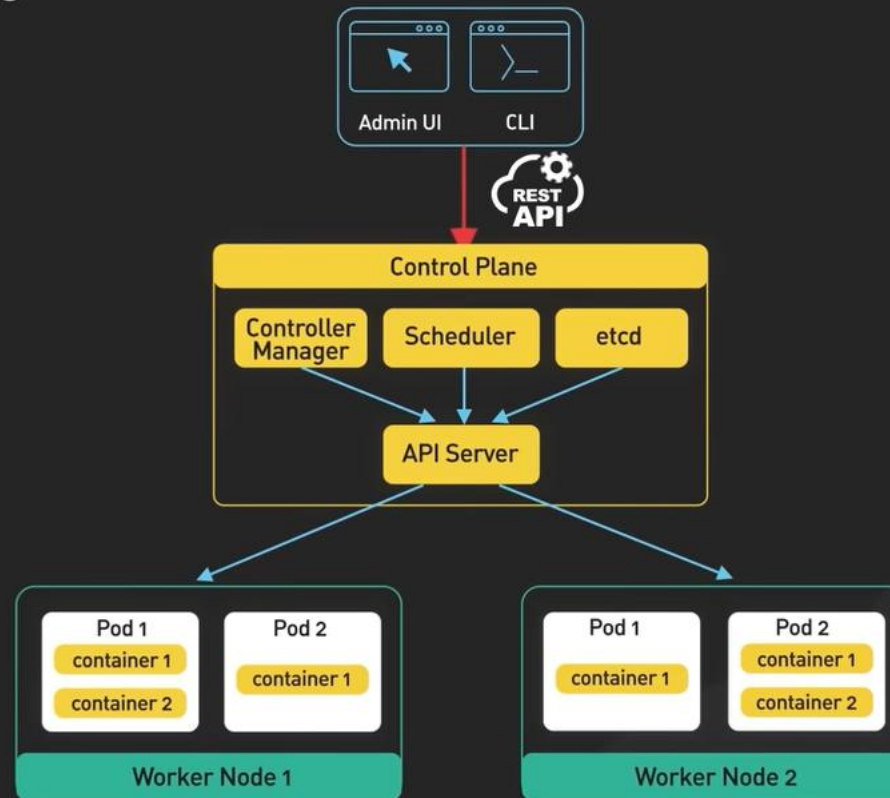
1:18 / 6:27 • Kubernetes Architecture >



Kubernetes Explained in 6 Minutes | k8s Architecture

Kubernetes in 6 mins

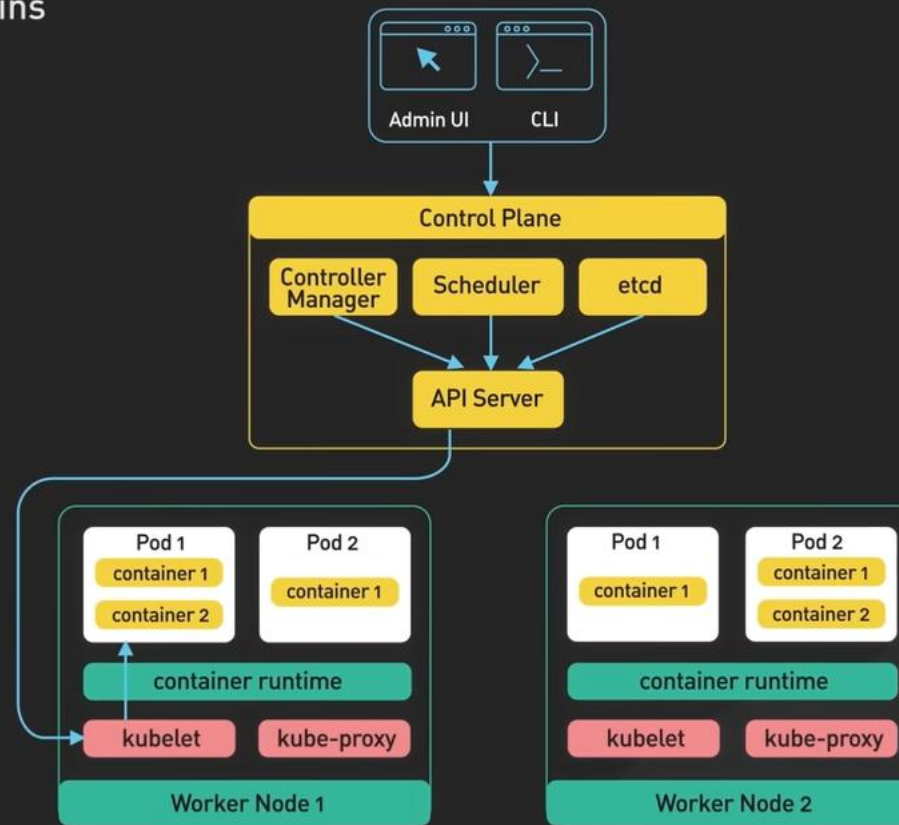
ByteByteGo.com



Kubernetes Explained in 6 Minutes | k8s Architecture

Kubernetes in 6 mins

ByteByteGo.com



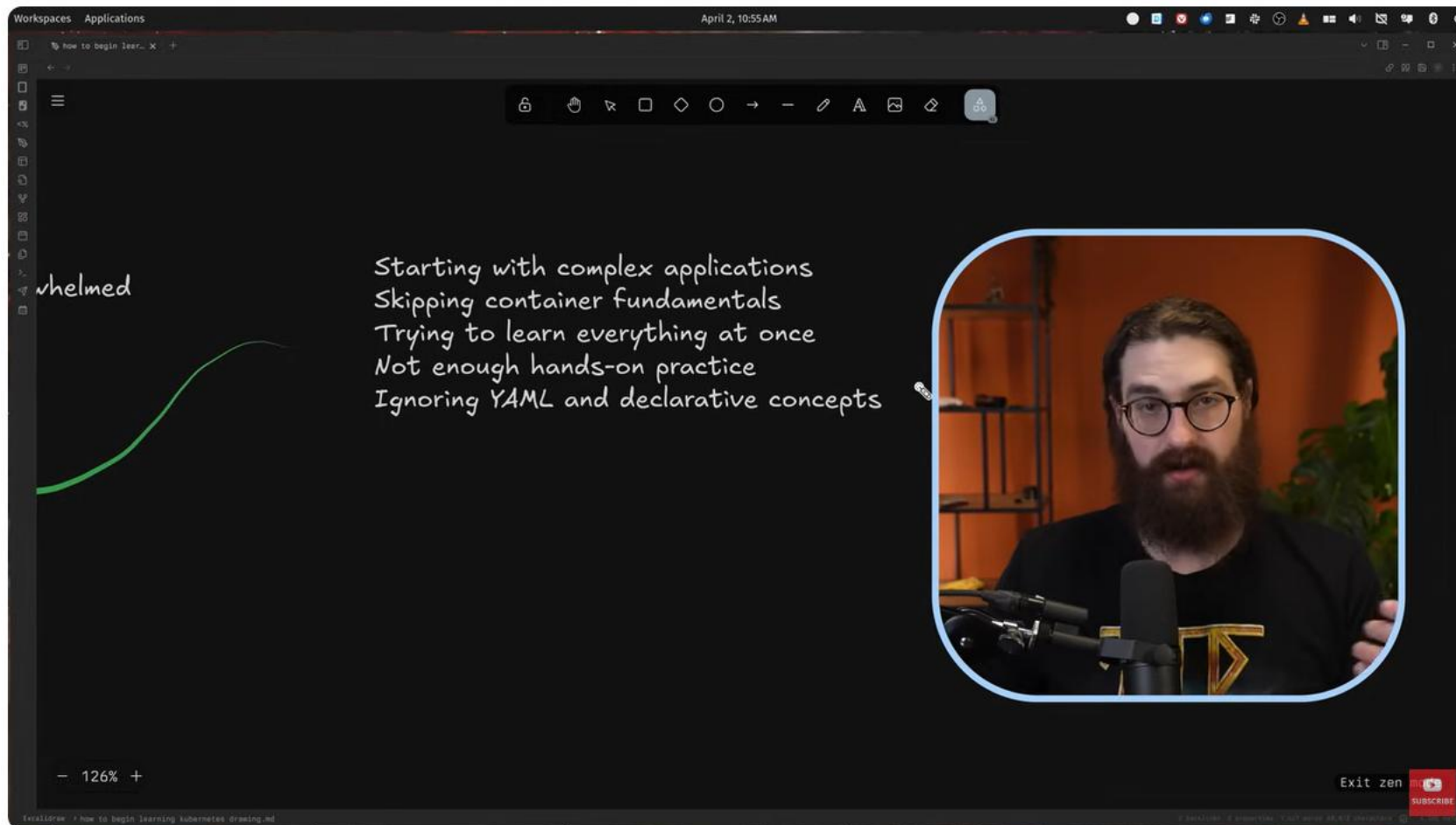
Kubernetes Explained in 6 Minutes | k8s Architecture

- etcd : stores key-value data
- highly scalable
- portable
- operating cost is high

- complex to manage
- costly

Comment apprendre Kubernetes en 2025

Common mistakes:



Comment apprendre Kubernetes en 2025

Prerequisites:

Workspaces Applications April 2, 10:57 AM

how to begin lear... X +

co

prereqs

Linux

- Command line
- File system
- Understanding processes
- Networking concepts
- YAML

Containers

- Container images and re
- Dockerfile basics
- Container lifecycle
- Container networking ba
- Persistent storage with

126% +

how to begin learning kubernetes drawing.md

SUBSCRIBE

Comment apprendre Kubernetes en 2025

Workspaces Applications April 2, 10:58 AM

how to begin learn... X +

Containers

Container images and registries
Dockerfile basics
Container lifecycle
Container networking basics
Persistent storage with containers

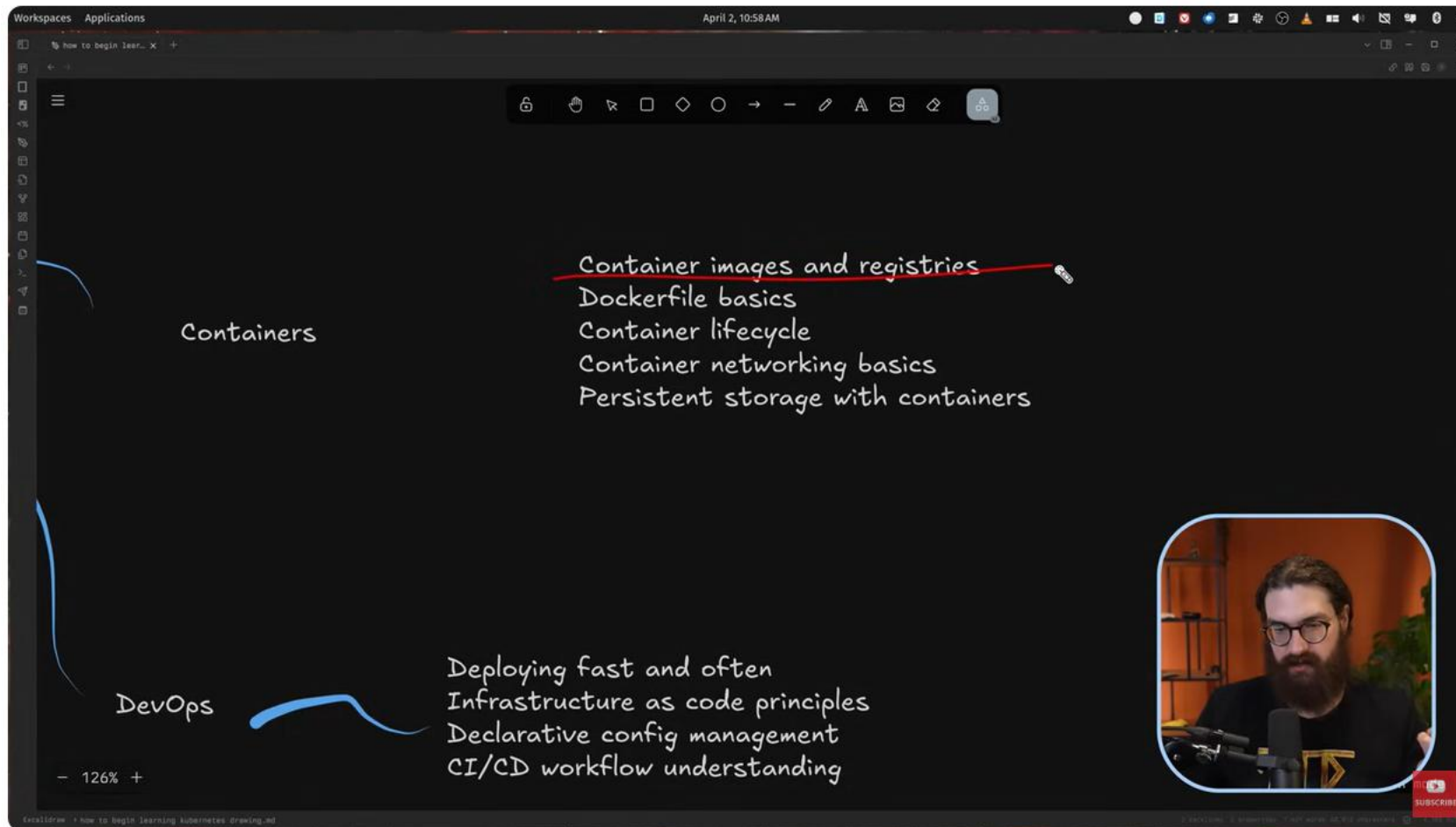
DevOps

Deploying fast and often
Infrastructure as code principles
Declarative config management
CI/CD workflow understanding

126% +

how to begin learning kubernetes drawing.md

SUBSCRIBE



Comment apprendre Kubernetes en 2025

- Github: <https://github.com/mischavandenburg>
- Learn docker/container
- Learning:
 - o <https://labs.iximiuz.com/>

- <https://www.skool.com/kubecraft/about>
- Tools:
 - Ranger desktop (open source)
 - k3s
- isolated environment to train

Cloud Native vs Cloud First vs Cloud Ready vs Born in the Cloud: The Ultimate Showdown

Cloud Ready	Cloud Native	Cloud First	Born in the Cloud
Cloud-ready are apps were not originally designed for the cloud but easily adapt to the cloud. <u>These apps are traditional applications that leverage cloud functionality.</u>	Cloud-native is a set of practices and a style of building and running applications that take advantages of the cloud computing model. <u>Cloud-native is about how applications are created and deployed, not where.</u>	Cloud-first is a strategic approach where organizations <u>prioritize the use of cloud computing services. It's about prioritizing the cloud over on-premises solutions whenever feasible.</u>	"Born in the cloud" applications are developed for and operated in the cloud since their inception. <u>Born-in-the-cloud apps start and grow without ever having physical data centers or legacy infrastructure.</u>
Cloud-ready applications may not use <u>microservices or modular architecture</u> , but have been modified to ensure compatibility with cloud environments. This can involve architectural changes to run on cloud infrastructure.	It emphasizes <u>microservices architecture</u> , where applications are broken down into <u>small, independent services</u> that communicate over <u>well-defined APIs</u> .	While it may leverage modern architectural patterns like microservices, the cloud-first strategy is more about the <u>decision-making process</u> regarding where to deploy and manage applications and data.	These applications and services are optimized cloud environments' scalability, flexibility, and <u>cost-efficiency</u> . They often use a <u>microservices architecture</u> , similar to cloud-native applications.
Cloud-ready applications can leverage cloud services like <u>virtual machines, containers, storage, and networking capabilities</u> .	Utilizes <u>containers, service meshes, microservices, immutable infrastructure, and declarative APIs</u> to make the application portable, scalable, and resilient.	Open to using a wide range of cloud services (IaaS, PaaS, SaaS) and tools offered by cloud providers to meet business requirements, generally prefers <u>PaaS and SaaS</u> .	Prefer serverless computing, <u>containers, and managed database services</u> . The choice of technology is guided by the need to <u>innovate quickly and scale efficiently</u> .
The focus is on compatibility and operational efficiency in the cloud, ensuring that <u>existing applications</u> can be hosted on cloud platforms without requiring a <u>complete redesign</u> .	The focus is on application development techniques that are native to cloud environments, aiming to enhance <u>scalability, flexibility, and resilience</u> .	The focus is on the cloud as the preferred platform for deploying applications and services, with the aim of achieving <u>cost efficiency, flexibility, and agility</u> at the organizational level.	The focus is on leveraging cloud technologies not just for <u>operational efficiency</u> but as a <u>fundamental driver of business models, product development, and competitive advantage</u> .



Cloud Native vs Cloud First vs Cloud Ready vs Born in the Cloud: The Ultimate Showdown

Migration

Migrating to Kubernetes + Best Practices for Cloud Native • T. Vitale & L. Højgaard • GOTO 2021

- k8s adds complexity, should be known before moving into k8s
- migration: make the application stateless/cloud native principles
- Ensure that the tests are complete/ready to ensure it is working once deployed
- Not always necessary to split a monolith into micro services if too much tangled/cobbled services, it would become a distributed monolith instead of micro-services
- With containers, sometimes DRY principle should not be too strict (e.g. copy paste into several micro-services for coupling)
- One application One container, Do Not migrate all at once
- Embrace the community and mindset

[Migrating to Kubernetes](#)

Learning

[Kubernetes Tutorial for Beginners \[FULL COURSE in 4 Hours\]](#)

Intro to K8s:

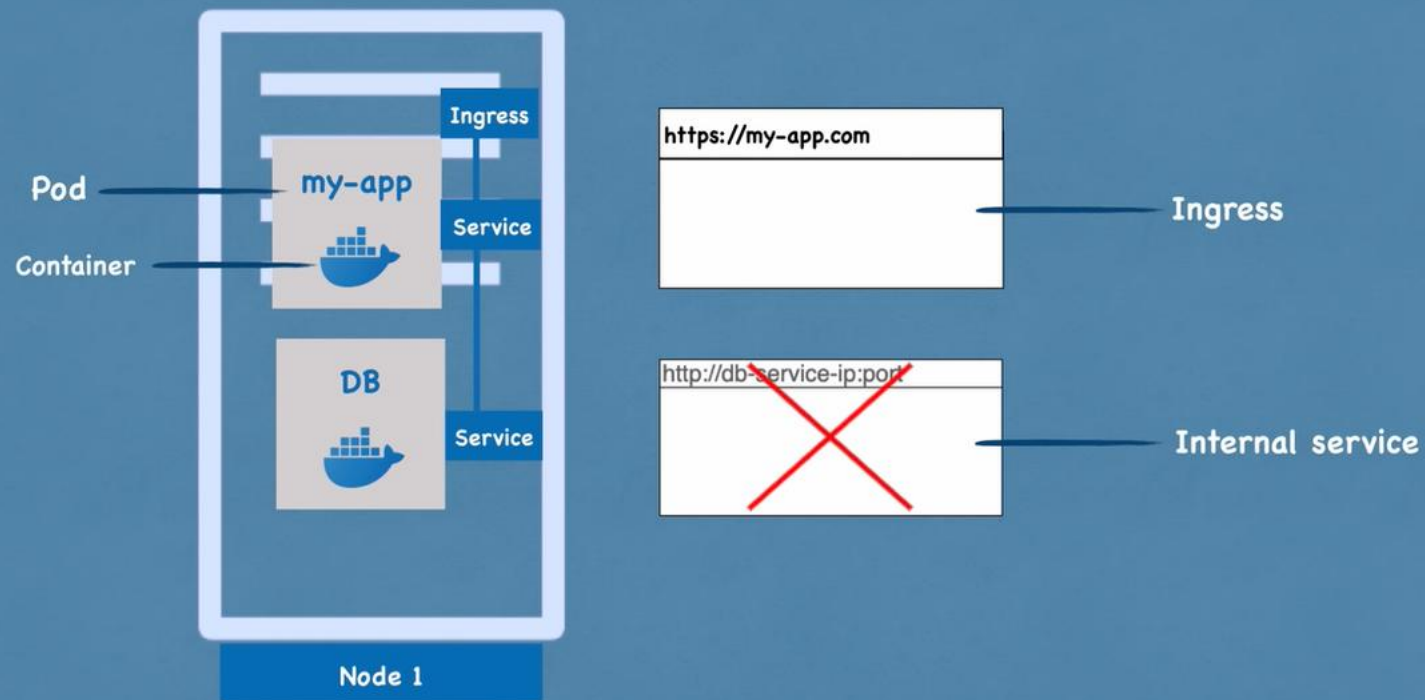
Overview 

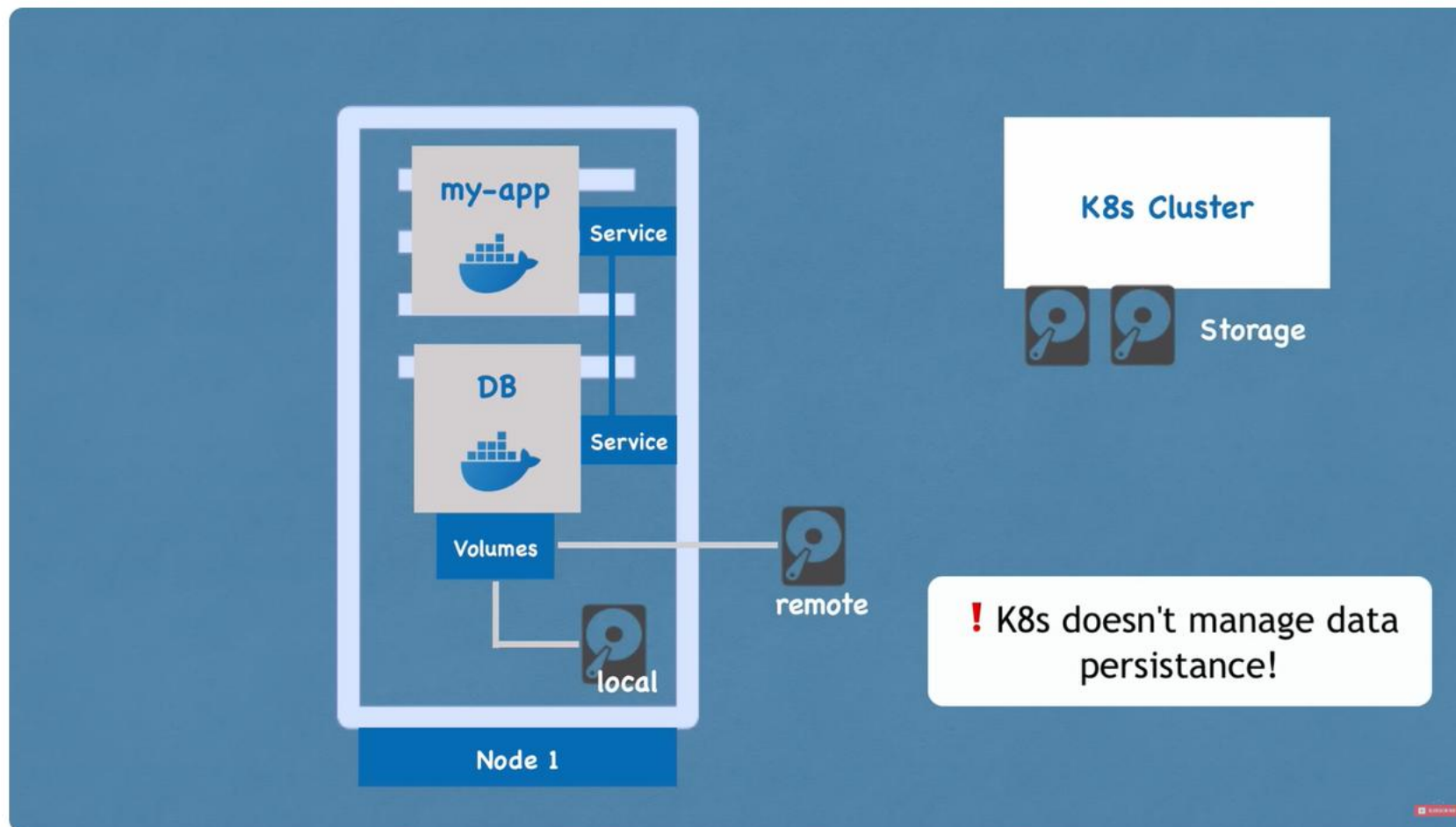
- ▶ What is Kubernetes?
- ▶ K8s Architecture
- ▶ Main K8s Components
- ▶ Minikube and Kubectl - Local Setup
- ▶ Main Kubectl Commands - K8s CLI
- ▶ K8s YAML Configuration File
- ▶ **Hands-On Demo**



 [Watch on YouTube](#)

Kubernetes Tutorial for Beginners [FULL COURSE in 4 Hours]

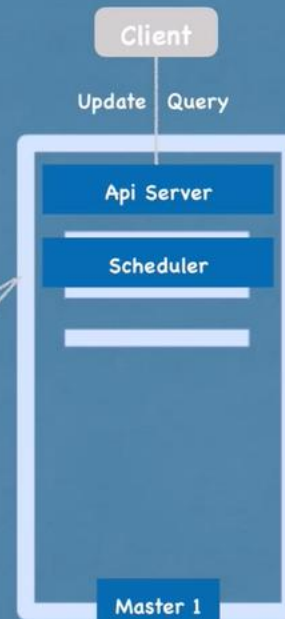
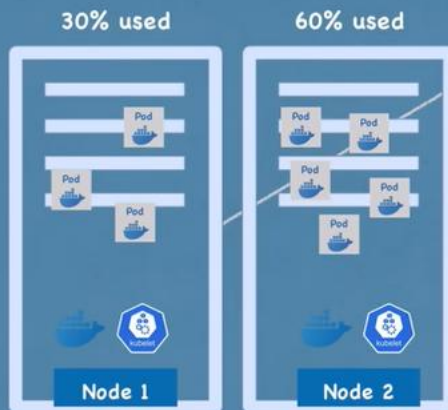






Master processes

Scheduler *just decides* on which Node new Pod should be scheduled



Schedule new Pod

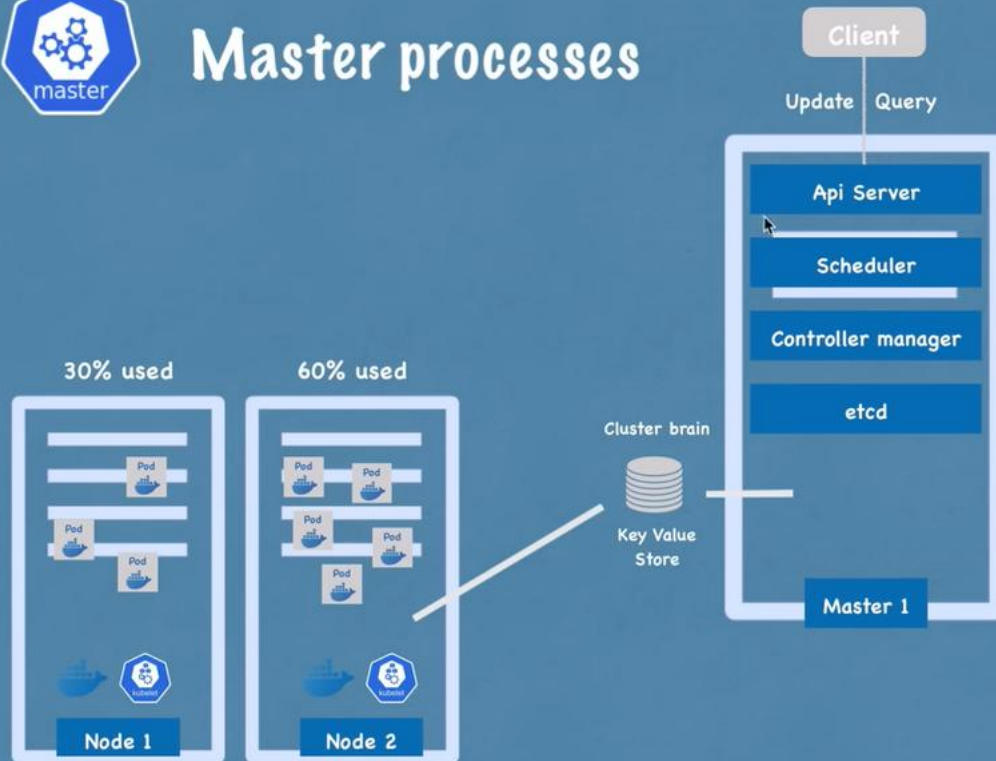
API Server

Scheduler

Where to put the Pod?

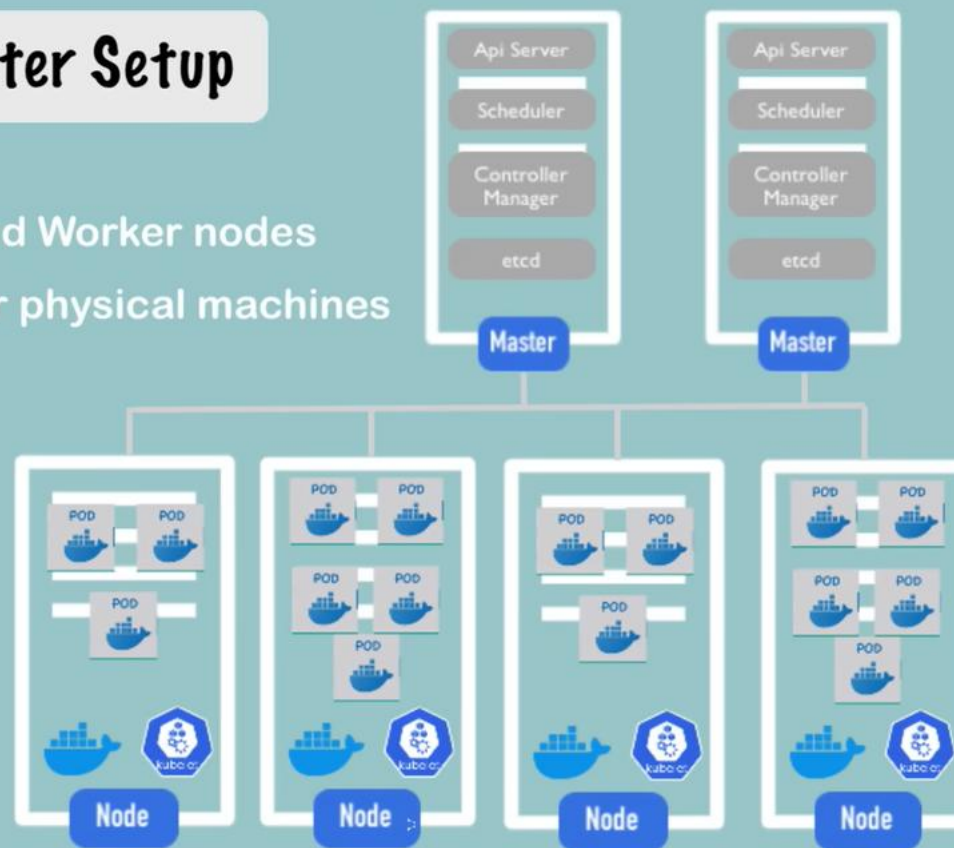


Master processes



Production Cluster Setup

- Multiple Master and Worker nodes
- Separate virtual or physical machines





```
kubectl create deployment nginx-depl --image=nginx
```

- **blueprint** for creating pods
- **most basic configuration** for deployment (name and image to use)
- **rest defaults**

Subscribe

Layers of Abstraction



Deployment manages a ..



ReplicaSet manages a ..



Pod is an abstraction of ..



Container



```
Terminal Shell Edit View Window Help
~ - bash - 89x24
[~]$ kubectl get pod
NAME                                READY   STATUS    RESTARTS   AGE
mongo-depl-67f895857c-fkspm        1/1     Running   0           3m5s
nginx-depl-66859c8f65-vfjjk       1/1     Running   0           11m
[~]$ kubectl exec -it mongo-depl-67f895857c-fkspm -- bin/bash
root@mongo-depl-67f895857c-fkspm:/# ls
bin      dev      home     lib64    opt      run      sys      var
boot     docker-entrypoint-initdb.d  js-yaml.js  media    proc     sbin     tmp
data     etc      lib      mnt      root     srv      usr
root@mongo-depl-67f895857c-fkspm:/# exit
exit
[~]$
```

Kubernetes Tutorial for Beginners [FULL COURSE in 4 Hours]

Debugging pods

Log to console

```
kubectl logs [pod name]
```

Get Interactive Terminal

```
kubectl exec -it [pod name] -- bin/bash
```

Use configuration file for CRUD

Apply a configuration file

```
kubectl apply -f [file name]
```

Delete with configuration file

```
kubectl delete -f [file name]
```



K8s updates state continuously!

```
! nginx-deployment.yaml x
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: nginx-deployment
5    labels: ...
7  spec:
8    replicas: 2
9    selector: ...
12 template: ...
22 status:
    availableReplicas: 1
    conditions:
      - lastTransitionTime: "2020-01-24T10:54:59Z"
        lastUpdateTime: "2020-01-24T10:54:59Z"
        message: Deployment has minimum availability.
        reason: MinimumReplicasAvailable
        status: "True"
        type: Available
      - lastTransitionTime: "2020-01-24T10:54:56Z"
        lastUpdateTime: "2020-01-24T10:54:59Z"
        message: ReplicaSet "nginx-deployment-7d64f4b
```

Template

```
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: nginx-deployment
5  >   labels: ...
7  spec:
8    replicas: 2
9  >   selector: ...
12   template:
13     metadata:
14       labels:
15         app: nginx
16     spec:
17       containers:
18         - name: nginx
19           image: nginx:1.16
20           ports:
21             - containerPort: 8080
22
```

- has it's own "metadata" and "spec" section
- applies to Pod

```
~/Documents — -bash — 89x24
[Documents]$ kubectl apply -f nginx-deployment.yaml
deployment.apps/nginx-deployment created
[Documents]$ kubectl apply -f nginx-service.yaml
service/nginx-service created
[Documents]$ kubectl get pod
NAME                                READY   STATUS    RESTARTS   AGE
nginx-deployment-7d64f4b574-fklxj   1/1     Running   0           10s
nginx-deployment-7d64f4b574-v7mwj   1/1     Running   0           10s
[Documents]$ kubectl get service
NAME            TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)    AGE
kubernetes      ClusterIP   10.96.0.1    <none>        443/TCP    2d
nginx-service   ClusterIP   10.96.25.229 <none>        80/TCP     19s
[Documents]$
```

1:11:30 / 3:36:54 • K8s YAML Configuration File >

Kubernetes Tutorial for Beginners [FULL COURSE in 4 Hours]

Overview of K8s Components

2 Deployment / Pod
2 Service
1 ConfigMap
1 Secret

Internal Service

ConfigMap
DB Url

Deployment.yaml

Env
variables

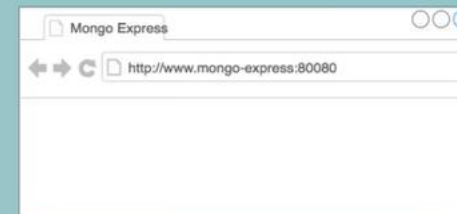


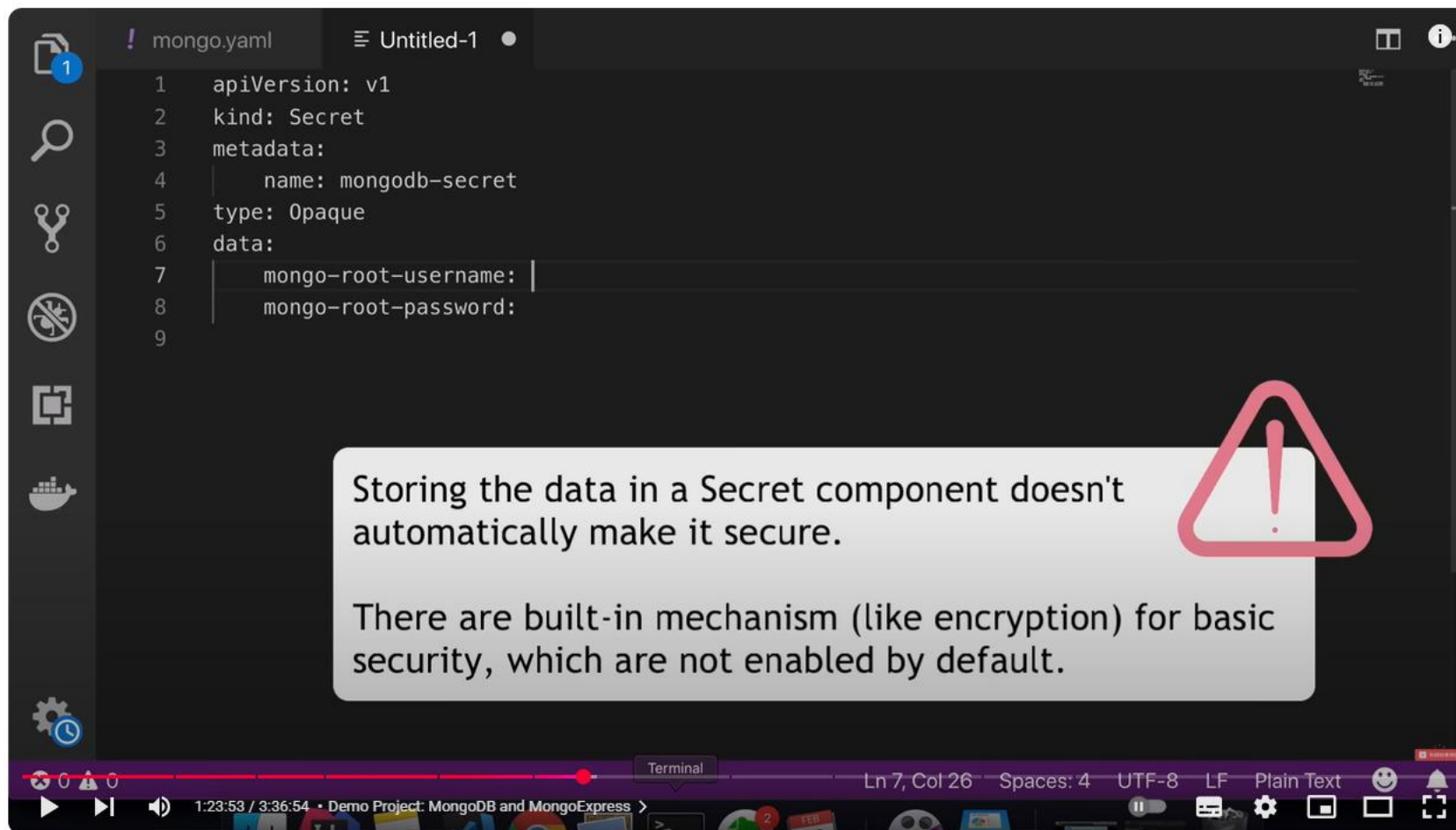
MongoDB

Secret
DB User
DB Pwd



Mongo Express





The screenshot shows a video player interface. The main content area displays a code editor with a file named `mongo.yaml`. The code is as follows:

```
1  apiVersion: v1
2  kind: Secret
3  metadata:
4    name: mongodb-secret
5  type: Opaque
6  data:
7    mongo-root-username: 
8    mongo-root-password: 
9
```

Overlaid on the code editor is a semi-transparent grey box containing the following text:

Storing the data in a Secret component doesn't automatically make it secure.

There are built-in mechanism (like encryption) for basic security, which are not enabled by default.

To the right of the text box is a red warning icon (a triangle with an exclamation mark). The video player's bottom bar shows a progress bar at 1:23:53 / 3:36:54, the title 'Demo Project: MongoDB and MongoExpress', and various control icons.

Kubernetes Tutorial for Beginners [FULL COURSE in 4 Hours]

```
~ - bash
~/Documents/A-Techworld-Nana/Youtube-Channel/K8S Tutorial/Material/mongodb-mongoexpress - bash
[~]$ kubectl get all
NAME                                TYPE                CLUSTER-IP      EXTERNAL-IP      PORT(S)
  AGE
service/kubernetes                 ClusterIP           10.96.0.1       <none>           443/TCP
  10m
[~]$ echo -n 'username' | base64
dXNlcm5hbWU=
[~]$
```

Kubernetes Tutorial for Beginners [FULL COURSE in 4 Hours]

The screenshot shows a video player interface with two main panes. The left pane displays a YAML file named `mongo-secret.yaml` with the following content:

```
21 - containerPort: 2701
22 env:
23 - name: MONGO_INITDB_
24   valueFrom:
25     secretKeyRef:
26       name: mongodb-s
27       key: mongo-root
28 - name: MONGO_INITDB_
29   valueFrom:
30     secretKeyRef:
31       name: mongodb-s
32       key: mongo-root
33 ---
34 apiVersion: v1
35 kind: Service
36 metadata:
37   name: mongodb-service
38 spec:
39   selector:
40     app: mongodb
41   ports:
```

The right pane, titled "Service Configuration File", lists the following configuration details:

- **kind:** "Service"
- **metadata / name:** a random name
- **selector:** to connect to Pod through label

The video player interface at the bottom shows a progress bar at 1:30:11 / 3:36:54, with a title "Dem..." and various control icons.

Kubernetes Tutorial for Beginners [FULL COURSE in 4 Hours]

The screenshot shows a code editor with four tabs: `mongo.yaml`, `mongo-express.yaml` (active), `mongo-configmap.yaml`, and `mongo-secret.yaml`. The active tab displays a Kubernetes Service manifest for `mongo-express` with the following configuration:

```
35     configMapKeyRef:
36       name: mongodb-configmap
37       key: database_url
38   ---
39   apiVersion: v1
40   kind: Service
41   metadata:
42     name: mongo-express-service
43   spec:
44     selector:
45       app: mongo-express
46     type: LoadBalancer
47     ports:
48       - protocol: TCP
49         port: 8081
50         targetPort: 8081
51         nodePort: 30000
```

The sidebar on the right contains the following text:

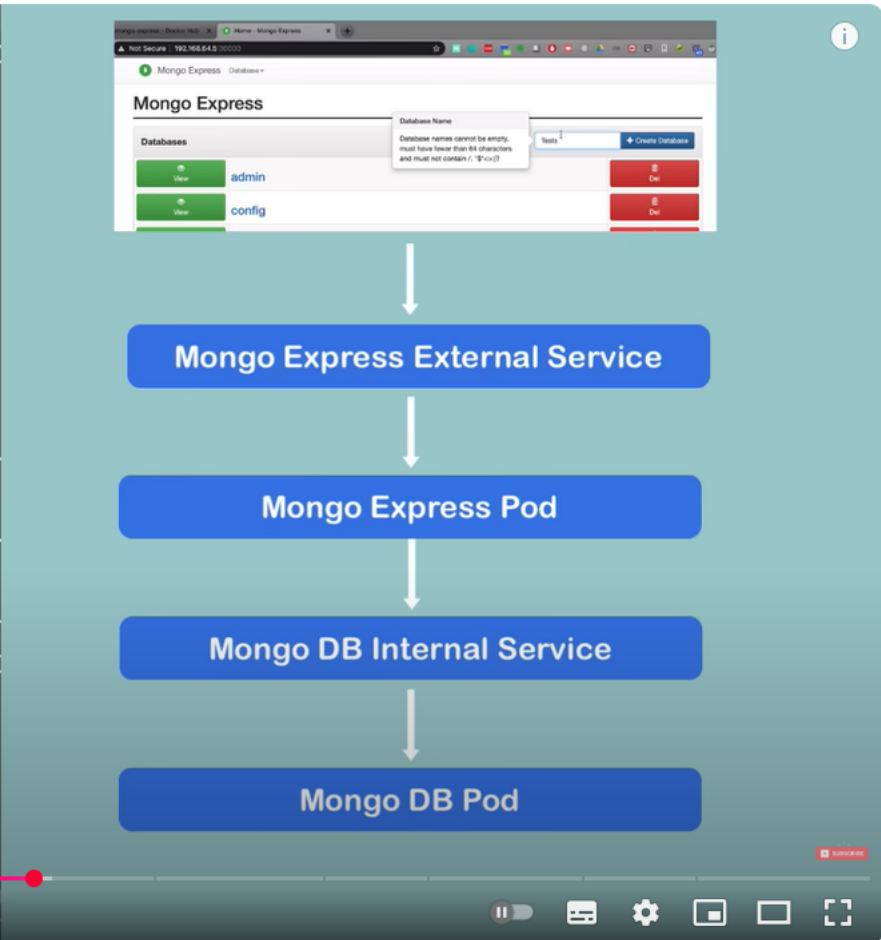
How to make it an External Service?

- `type:` "Loadbalancer"
..assigns service an external IP address and so accepts external requests
- `nodePort` must be between 30000-32767

The status bar at the bottom indicates the cursor is at `Ln 51, Col 22`, with `Spaces: 2`, `UTF-8`, `LF`, and `YAML` encoding.

Kubernetes Tutorial for Beginners [FULL COURSE in 4 Hours]

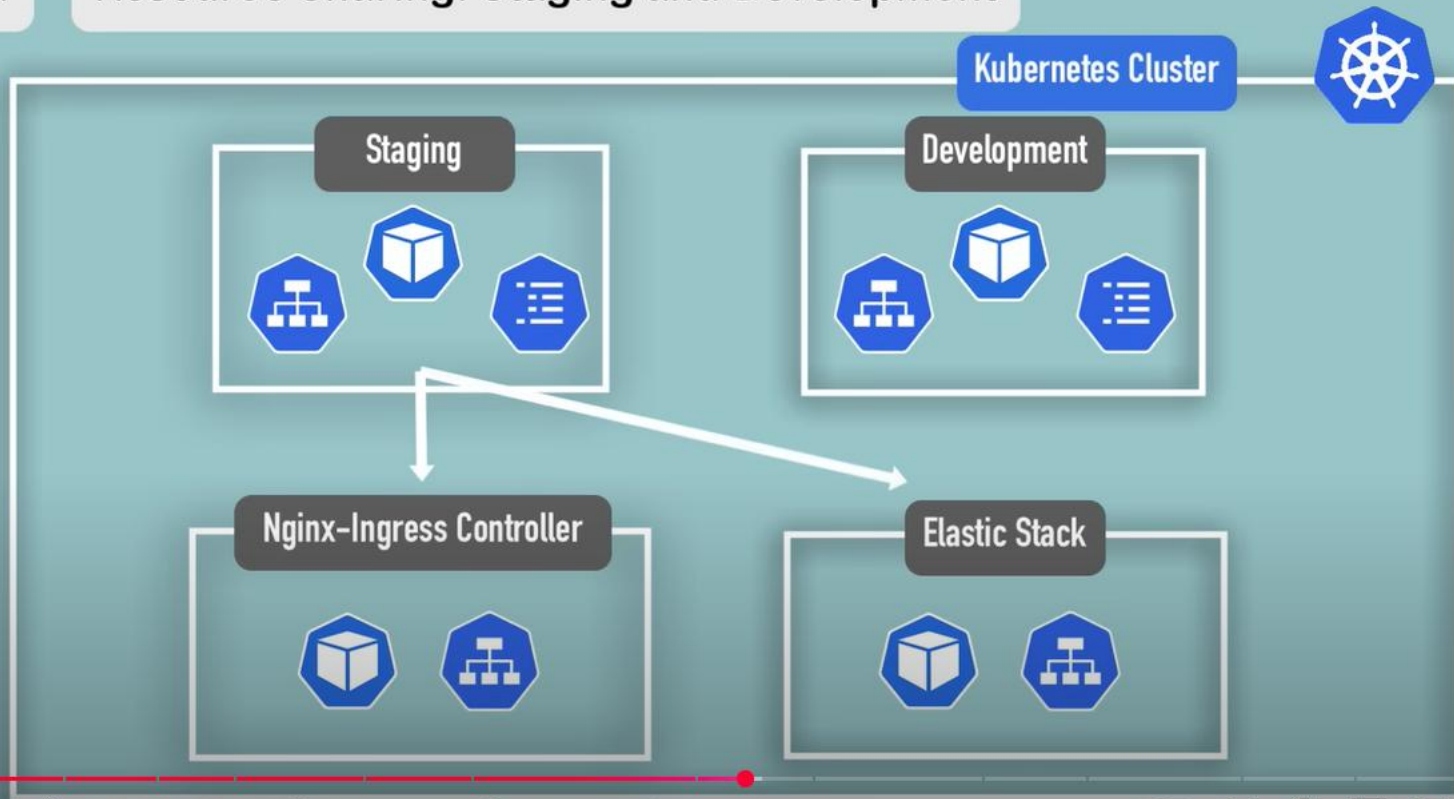

```
~/k8s-configuration ~ -bash
[k8s-configuration]$ kubectl get service
NAME                                TYPE
kubernetes                          ClusterIP
mongo-express-service               LoadBalancer
mongodb-service                     ClusterIP
[k8s-configuration]$ minikube service
-----
| NAMESPACE | NAME |
|-----|-----|
| default   | mongo-express-service |
|-----|-----|
Opening service default/mongo-express
[k8s-configuration]$
```



Kubernetes Tutorial for Beginners [FULL COURSE in 4 Hours]

3.

Resource Sharing: Staging and Development



1:52:48 / 3:36:54 • Organizing your components with K8s Namespaces >



Kubernetes Tutorial for Beginners [FULL COURSE in 4 Hours]



Use Cases when to use Namespaces

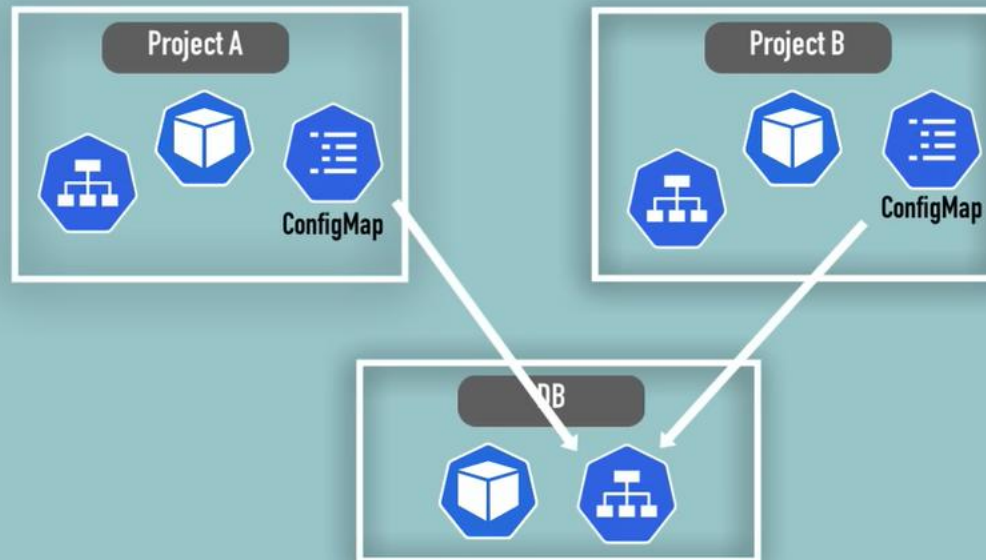
1. **Structure** your components
2. **Avoid conflicts** between teams
3. **Share services** between different environments
4. **Access and Resource Limits** on Namespaces Level



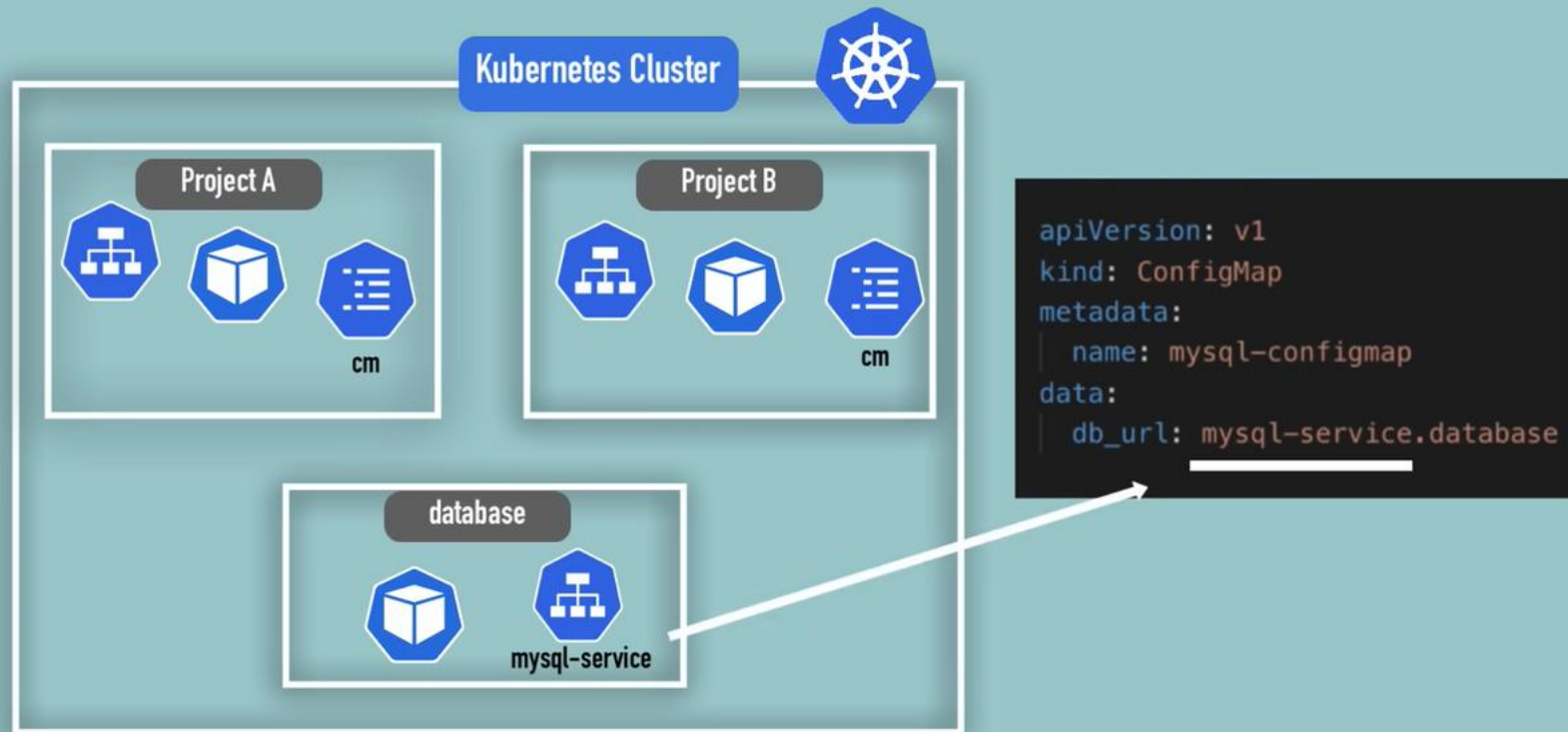
You can't access most resources from another Namespace

Each NS must define own ConfigMap

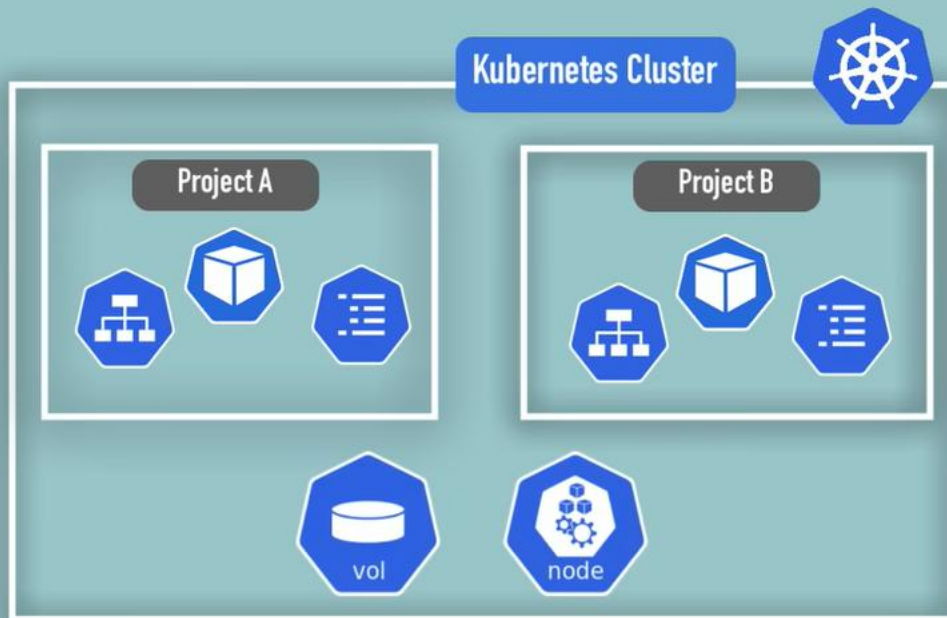
Kubernetes Cluster



Access Service in another Namespace



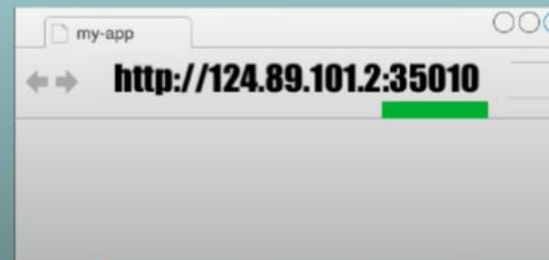
Components, which can't be created within a Namespace



- live globally in a cluster
- you can't isolate them

Example YAML File: External Service

```
apiVersion: v1
kind: Service
metadata:
  name: myapp-external-service
spec:
  selector:
    app: myapp
  type: LoadBalancer
  ports:
    - protocol: TCP
      port: 8080
      targetPort: 8080
      nodePort: 35010
```

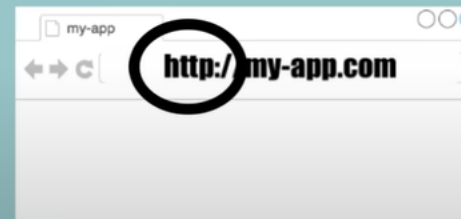


Example YAML File: Ingress

```
apiVersion: networking.k8s.io/v1beta1
kind: Ingress
metadata:
  name: myapp-ingress
spec:
  rules:
  - host: myapp.com
    http:
      paths:
      - backend:
          serviceName: myapp-internal-service
          servicePort: 8080
```

Routing rules:

Forward request to the
internal service.



Nothing configured for
https yet!

2:05:03 / 3:36:54 • K8s Ingress explained >



Kubernetes Tutorial for Beginners [FULL COURSE in 4 Hours]

Ingress and Internal Service configuration

Ingress:

```
apiVersion: networking.k8s.io/v1beta1
kind: Ingress
metadata:
  name: myapp-ingress
spec:
  rules:
  - host: myapp.com
    http:
      paths:
      - backend:
          serviceName: myapp-internal-service
          servicePort: 8080
```

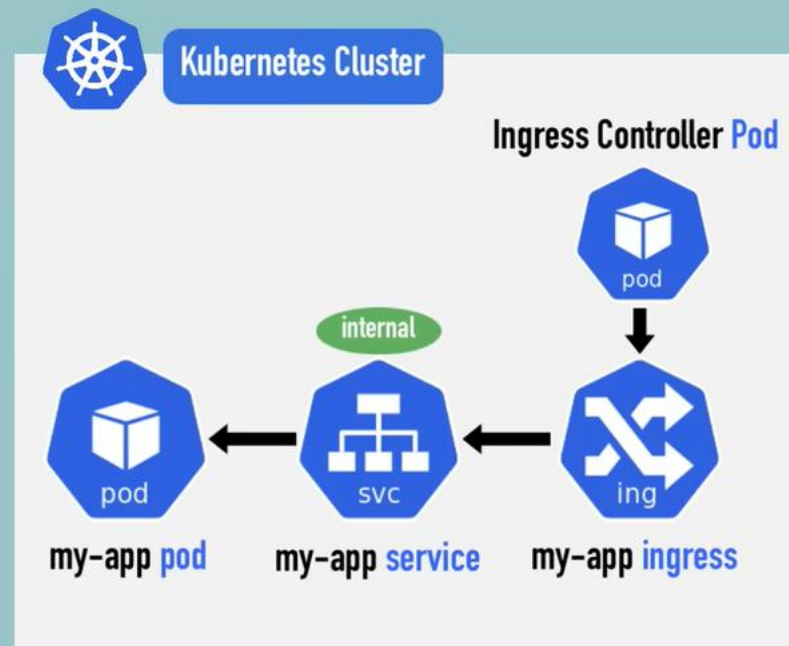
Internal Service:

```
apiVersion: v1
kind: Service
metadata:
  name: myapp-internal-service
spec:
  selector:
    app: myapp
  ports:
  - protocol: TCP
    port: 8080
    targetPort: 8080
```



© 2020 K8S

What is Ingress Controller? 🤔



- evaluates all the rules
- manages redirections
- entrypoint to cluster
- many third-party implementations
- K8s Nginx Ingress Controller



Multiple paths for same host

```
apiVersion: networking.k8s.io/v1beta1
kind: Ingress
metadata:
  name: simple-fanout-example
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  rules:
  - host: myapp.com
    http:
      paths:
      - path: /analytics
        backend:
          serviceName: analytics-service
          servicePort: 3000
      - path: /shopping
        backend:
          serviceName: shopping-service
          servicePort: 8080
```

- Example Google
- One domain but many services

Multiple sub-domains or domains

```
apiVersion: networking.k8s.io/v1beta1
kind: Ingress
metadata:
  name: name-virtual-host-ingress
spec:
  rules:
  - host: analytics.myapp.com
    http:
      paths:
        backend:
          serviceName: analytics-service
          servicePort: 3000
  - host: shopping.myapp.com
    http:
      paths:
        backend:
          serviceName: shopping-service
          servicePort: 8080
```

Instead of:

<http://myapp.com/analytics>

<http://analytics.myapp.com>

© 2020

Multiple sub-domains or domains

```
apiVersion: networking.k8s.io/v1beta1
kind: Ingress
metadata:
  name: name-virtual-host-ingress
spec:
  rules:
  - host: analytics.myapp.com
    http:
      paths:
        backend:
          serviceName: analytics-service
          servicePort: 3000
  - host: shopping.myapp.com
    http:
      paths:
        backend:
          serviceName: shopping-service
          servicePort: 8080
```

```
spec:
  rules:
  - host: myapp.com
    http:
      paths:
        - path: /analytics
          backend:
            serviceName: analytics-service
            servicePort: 3000
        - path: /shopping
          backend:
            serviceName: shopping-service
            servicePort: 8080
```

Instead of **1 host** and **multiple path**

You have now **multiple hosts** with **1 path**. Each host represents a subdomain.

Configuring TLS Certificate - https//



```
apiVersion: networking.k8s.io/v1beta1
kind: Ingress
metadata:
  name: tls-example-ingress
spec:
  tls:
  - hosts:
    - myapp.com
    secretName: myapp-secret-tls
  rules:
  - host: myapp.com
    http:
      paths:
      - path: /
        backend:
          serviceName: myapp-internal-service
          servicePort: 8080
```

- tls

- secretName



Configuring TLS Certificate - https//



```
apiVersion: networking.k8s.io/v1beta1
kind: Ingress
metadata:
  name: tls-example-ingress
spec:
  tls:
  - hosts:
    - myapp.com
    secretName: myapp-secret-tls
  rules:
  - host: myapp.com
    http:
      paths:
      - path: /
        backend:
          serviceName: myapp-internal-service
          servicePort: 8080
```

A blue octagonal icon with a white 'X' and the word 'ing' below it, representing an Ingress resource.

```
apiVersion: v1
kind: Secret
metadata:
  name: myapp-secret-tls
  namespace: default
data:
  tls.crt: base64 encoded cert
  tls.key: base64 encoded key
type: kubernetes.io/tls
```

A blue octagonal icon with a white padlock and the word 'secret' below it, representing a Secret resource.

Kubernetes Tutorial for Beginners [FULL COURSE in 4 Hours]

Helm: package management for Kubernetes

Sharing Helm Charts 🕶️



Kubernetes Cluster

Need some Deployment?



pod

my-app pod



svc

my-app service

```
helm search <keyword>
```

or Helm Hub:

Discover & launch great
Kubernetes-ready apps

Search charts...

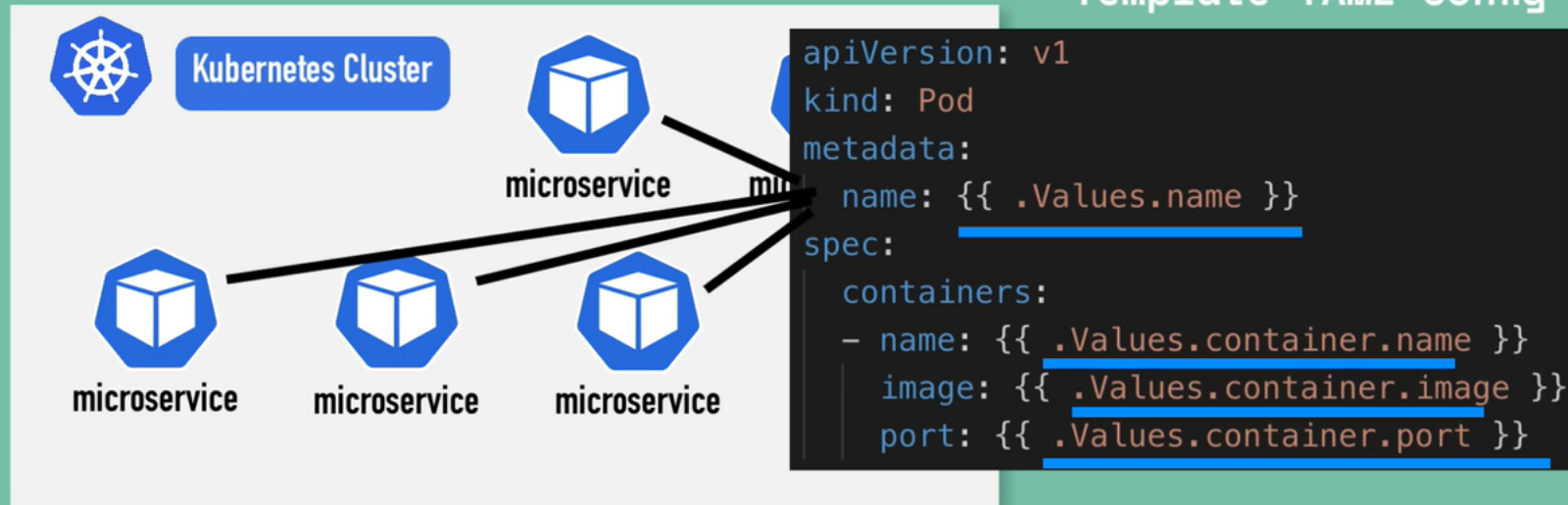
1140 charts ready to deploy

Kubernetes Tutorial for Beginners [FULL COURSE in 4 Hours]

Helm is also a templating Engine

Templating Engine

Template YAML config



`{{ .Values... }}`

Templating Engine

Template YAML config

values.yaml

`{{ .Values... }}`

```
name: my-app
container:
  name: my-app-container
  image: my-app-image
  port: 9001
```

```
apiVersion: v1
kind: Pod
metadata:
  name: {{ .Values.name }}
spec:
  containers:
    - name: {{ .Values.container.name }}
      image: {{ .Values.container.image }}
      port: {{ .Values.container.port }}
```

Values defined either via yaml file or with --set flag

Kubernetes Tutorial for Beginners [FULL COURSE in 4 Hours]

Helm: release management

Persistent Volume



- a cluster resource
- created via YAML file
 - kind: PersistentVolume
 - spec: e.g. how much storage?

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-name
spec:
  capacity:
    storage: 5Gi
  volumeMode: Filesystem
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Recycle
  storageClassName: slow
```

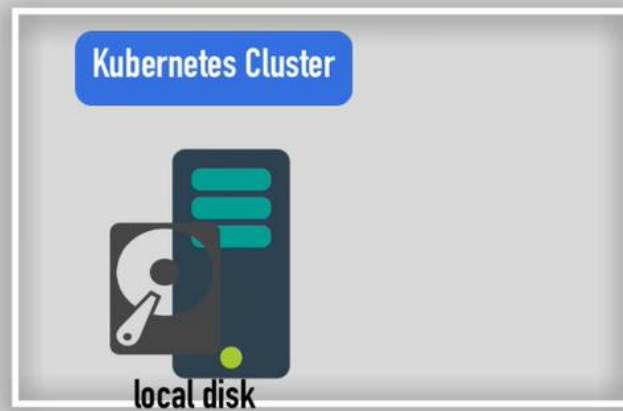


2:40:46 / 3:36:54 • Persisting Data in K8s with Volumes >

Persistent Volume



Needs actual physical storage, like



StatefulSet for stateful applications

stateful applications

- examples of stateful applications:



stateless applications

- don't keep record of state
- each request is completely new

Deployment of stateful and stateless applications

stateless
applications

deployed using Deployment



stateful
applications

deployed using StatefulSet



Deployment vs StatefulSet



mysql



mysql



mysql

more difficult

- ▶ can't be created/deleted at same time
- ▶ can't be randomly addressed
- ▶ replica Pods are not identical
 - **Pod Identity**

Pod Identity

- sticky identity for each pod



ID-0



ID-1



ID-2

- created from **same specification**, but **not interchangeable!**
- persistent identifier across any re-scheduling

Scaling database applications

MASTER



mysql-0

WORKER



mysql-1

WORKER



mysql-2

They do not use the same physical storage!

PV



/data/vol/pv-0

PV



/data/vol/pv-1

PV



/data/vol/pv-2

Kubernetes Services



- ▶ What is a Kubernetes Service and when we need it?
- ▶ Different Service types explained



ClusterIP Services



Headless Services



NodePort Services



LoadBalancer Services

- ▶ Differences between them and when to use which one

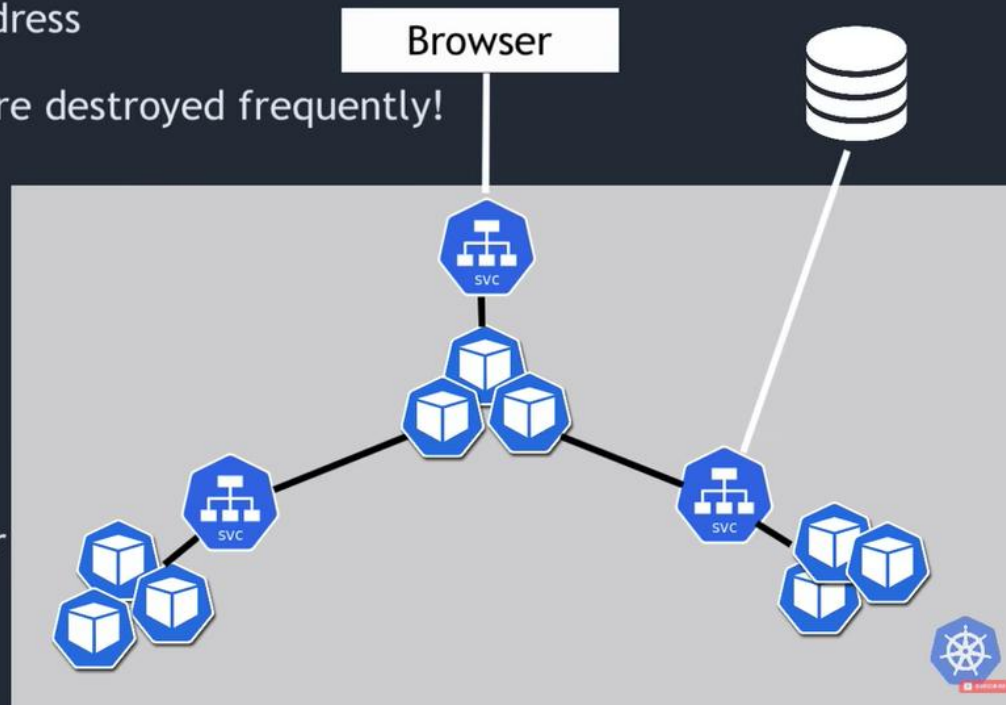
What is a Service and when we need it? 🤔

- ▶ Each Pod has its own IP address

❌ Pods are ephemeral - are destroyed frequently!

- ▶ Service:

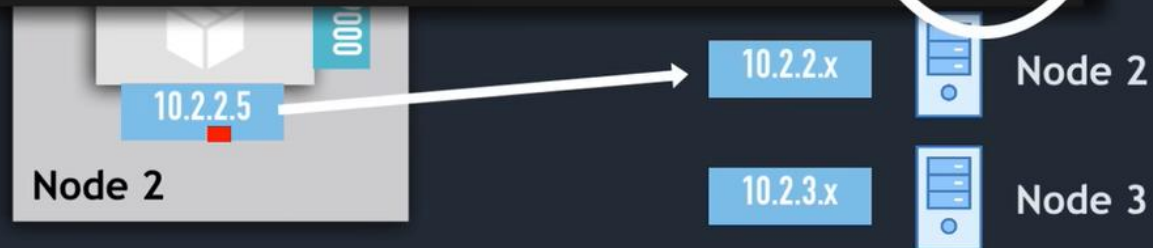
- ✅ stable IP address
- ✅ loadbalancing
- ✅ loose coupling
- ✅ within & outside cluster



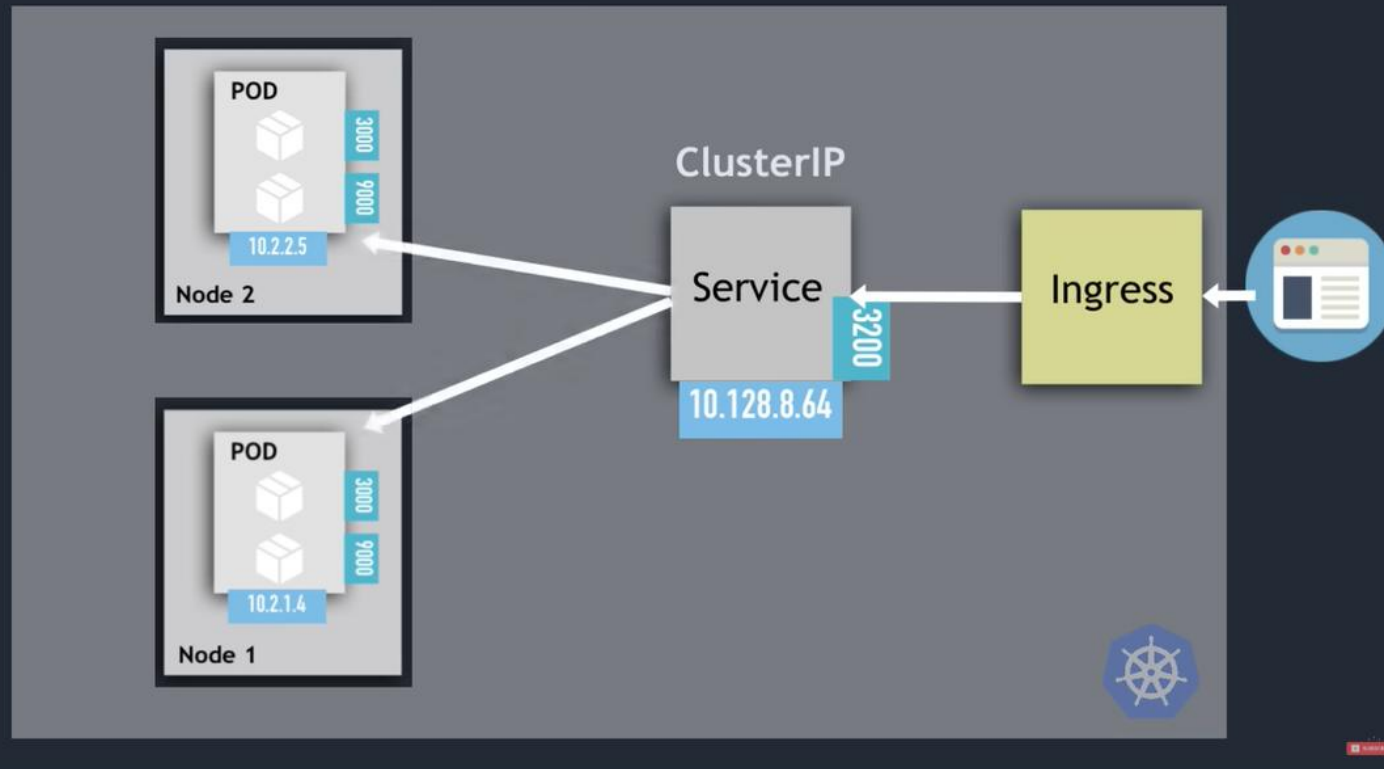
ClusterIP Services

```
kubectl get pod -o wide
```

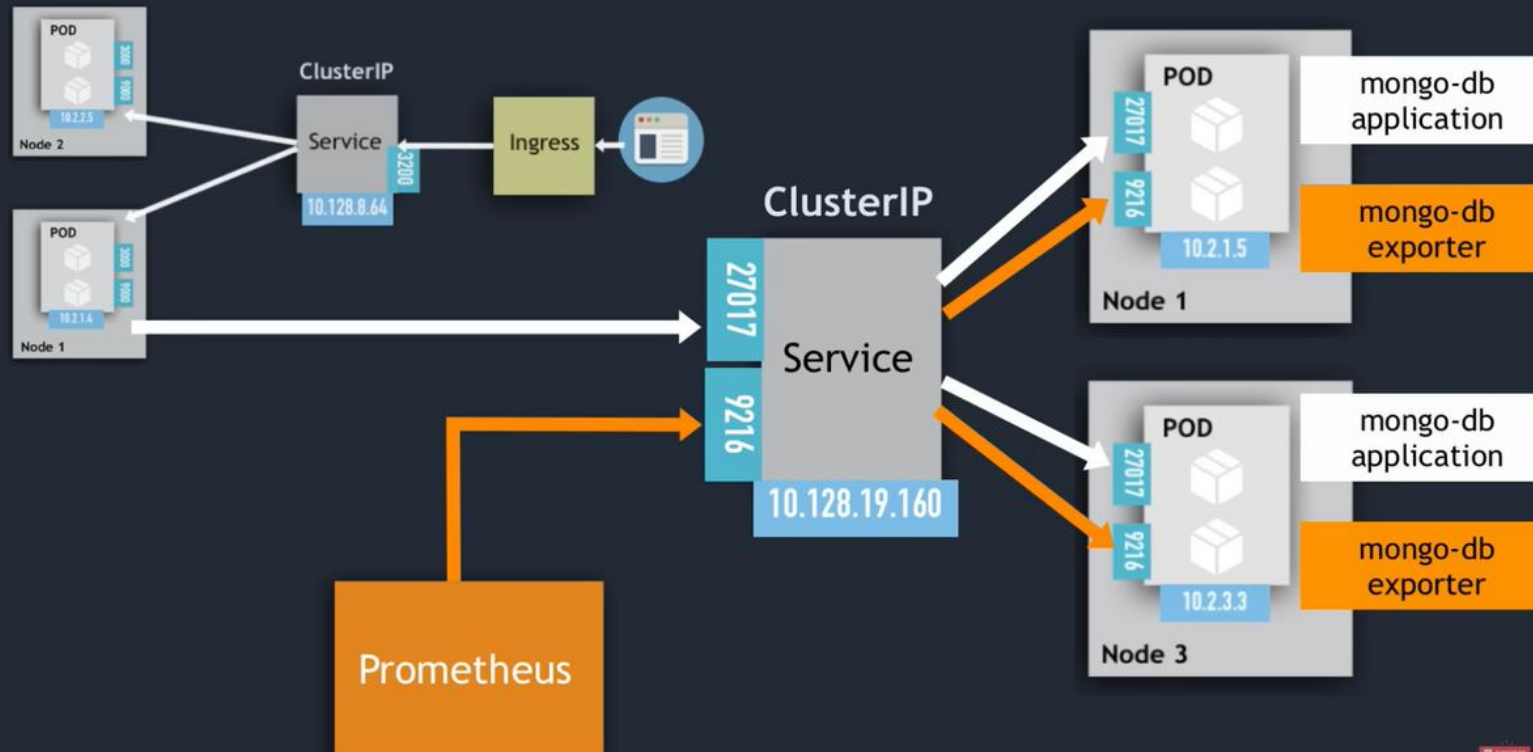
NAME	READY	STATUS	RESTARTS	AGE	IP
mongodb-deployment-55577c4cbc-gkz8b	1/1	Running	0	29s	10.2.2.2
mongodb-deployment-55577c4cbc-z8281	1/1	Running	0	29s	10.2.1.4
mongodb-deployment-55577c4cbc-zv9h8	1/1	Running	0	29s	10.2.2.3



Service Communication: selector



Multi-Port Services

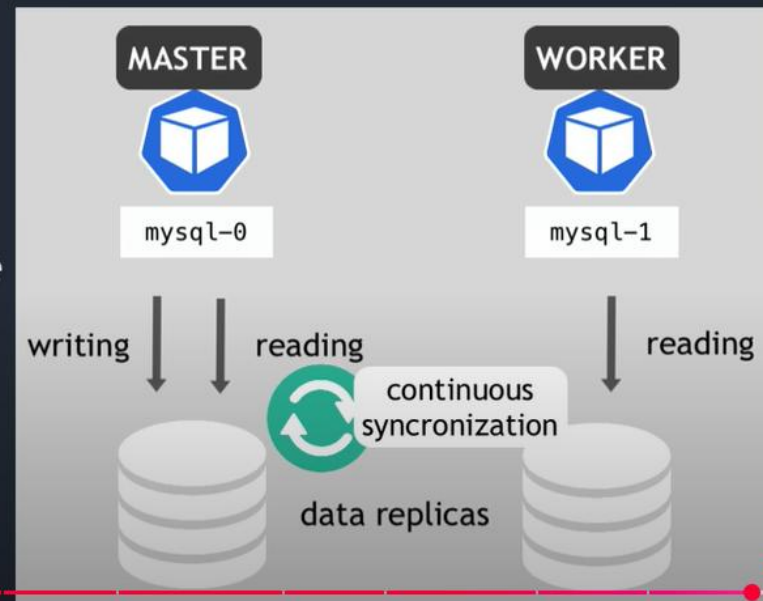


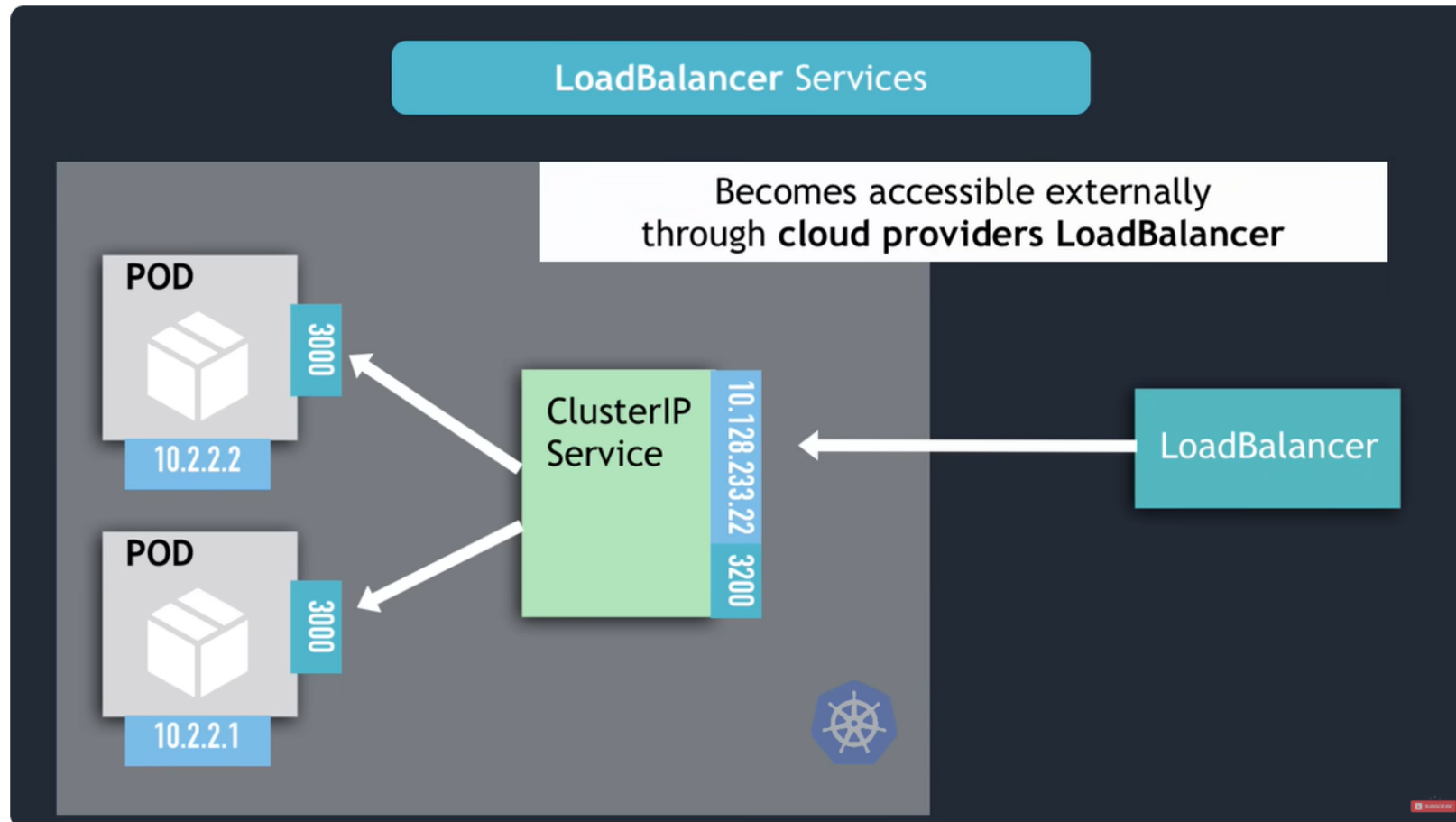
Headless Services

- ▶ Client wants to communicate with 1 specific Pod directly
- ▶ Pods want to talk directly with specific Pod
- ▶ So, not randomly selected
- ▶ Use Case: Stateful applications, like

✗ Pod replicas are not identical

Only Master is allowed to write to DB





Kubernetes Tutorial for Beginners [FULL COURSE in 4 Hours]

- <https://gitlab.com/nanuchi/youtube-tutorial-series/-/tree/master>
- Availability / Scalability / Restore & recover
- DB URL usually in the built application
- Data must be managed (backup, recover, etc.), k8s do Not manage them

- StatefulSet used for stateful application like database
- Worker Nodes are mandatory
- Must be installed in every node
 - Container runtime
 - Kubelet
 - Kube proxy
- Master Nodes
 - Api server (load balanced between all master nodes)
 - Through UI, API and/or CLI (kubectl)
 - Scheduler
 - Controller manager
 - etcd (distributed to all master nodes)
- on local machine, how to?
 - Use minikube
 - 1 node k8s cluster
 - Virtual box
 - Docker pre-installed
 - For testing purposes
- kubectl
- master node ~ control-plane rôle ?

```
v_tu@PN-N-0237:~$ minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured

v_tu@PN-N-0237:~$ kubectl get nodes
NAME        STATUS    ROLES    AGE   VERSION
minikube    Ready     control-plane  13m   v1.33.1

v_tu@PN-N-0237:~$ kubectl get pod
No resources found in default namespace.

v_tu@PN-N-0237:~$ kubectl get services
NAME         TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)    AGE
kubernetes   ClusterIP   10.96.0.1     <none>         443/TCP    17m

v_tu@PN-N-0237:~$
```

- In practice we do not handle pods directly
 - o But its abstraction: deployment
- Need image, exemple nginx

```
v_tu@PN-N-0237:~$ kubectl create deployment nginx-depl --image=nginx
deployment.apps/nginx-depl created
v_tu@PN-N-0237:~$ kubectl get deployment
NAME        READY   UP-TO-DATE   AVAILABLE   AGE
nginx-depl  0/1     1            0           10s

v_tu@PN-N-0237:~$ kubectl get pod
NAME                                READY   STATUS    RESTARTS   AGE
nginx-depl-68c944fcbc-crxhv        1/1     Running   0           20s

v_tu@PN-N-0237:~$ kubectl get pod
NAME                                READY   STATUS    RESTARTS   AGE
nginx-depl-68c944fcbc-crxhv        1/1     Running   0           51s

v_tu@PN-N-0237:~$ kubectl get deployment
NAME        READY   UP-TO-DATE   AVAILABLE   AGE
nginx-depl  1/1     1            1           59s

v_tu@PN-N-0237:~$ kubectl get replicaset
NAME                                DESIRED   CURRENT   READY   AGE
nginx-depl-68c944fcbc              1         1         1       2m34s

v_tu@PN-N-0237:~$ |
```

- Replicas are handled automatically; we don't handle them at all
- `$ kubectl edit deployment nginx-depl`

```

# Please edit the object below. Lines beginning with a '#' will be ignored,
# and an empty file will abort the edit. If an error occurs while saving this file will be
# reopened with the relevant failures.
#
apiVersion: apps/v1
kind: Deployment
metadata:
  annotations:
    deployment.kubernetes.io/revision: "1"
  creationTimestamp: "2025-05-28T09:21:41Z"
  generation: 1
  labels:
    app: nginx-depl
  name: nginx-depl
  namespace: default
  resourceVersion: "2051"
  uid: 0f1b45b3-525a-4d82-9798-53ec64a921b7
spec:
  progressDeadlineSeconds: 600
  replicas: 1
  revisionHistoryLimit: 10
  selector:
    matchLabels:
      app: nginx-depl
  strategy:
    rollingUpdate:
      maxSurge: 25%
      maxUnavailable: 25%
    type: RollingUpdate
  template:
    metadata:
      creationTimestamp: null
      labels:
        app: nginx-depl
    spec:
      containers:
      - image: nginx
        imagePullPolicy: Always
        name: nginx
        resources: {}
        terminationMessagePath: /dev/termination-log
        terminationMessagePolicy: File
      dnsPolicy: ClusterFirst
      restartPolicy: Always
      schedulerName: default-scheduler
      securityContext: {}
      terminationGracePeriodSeconds: 30
status:
  availableReplicas: 1
  conditions:

```

"/tmp/kubectrl-edit-2425763798.yaml" 66L, 1797B

- Autogenerated file (default since we didn't specify anything)
 - o kubectl create deployment <name of the deployment> --image=<name of the image>
 - o kubectl get pod
 - o kubectl logs
 - o kubectl describe pod <name of the pod>
- delete deployment (delete pod and replicas)

```
v_tu@PN-N-0237:~$ kubectl delete deployment mongo-depl
deployment.apps "mongo-depl" deleted
v_tu@PN-N-0237:~$ kubectl get pod
NAME                                READY   STATUS    RESTARTS   AGE
nginx-depl-68c944fcbc-crxhv         1/1     Running   0           17m
v_tu@PN-N-0237:~$
```

- Configuration

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.16
        ports:
        - containerPort: 80
```

o

```

v_tu@PN-N-0237:~$ touch nginx-deployment.yaml
v_tu@PN-N-0237:~$ vim nginx-deployment.yaml
v_tu@PN-N-0237:~$ kubectl apply -f nginx-deployment.yaml
deployment.apps/nginx-deployment created
v_tu@PN-N-0237:~$ ^C
v_tu@PN-N-0237:~$ kubectl get pod

```

NAME	READY	STATUS	RESTARTS	AGE
nginx-depl-68c944fcbc-crxhv	1/1	Running	0	27m
nginx-deployment-7f65fcf556-7s62t	1/1	Running	0	30s

```

v_tu@PN-N-0237:~$

```

-
- 3 parts
 - Metadata
 - Specification
 - Status (will be completed by k8s: Desired vs Actual)

- Service

```

apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  selector:
    app: nginx
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8080

```

○

```

v_tu@PN-N-0237:~$ vim nginx-deployment.yaml
v_tu@PN-N-0237:~$ touch nginx-service.yaml
v_tu@PN-N-0237:~$ vim nginx-service.yaml
v_tu@PN-N-0237:~$ vim nginx-service.yaml
v_tu@PN-N-0237:~$ kubectl apply -f nginx-deployment.yaml
deployment.apps/nginx-deployment unchanged
v_tu@PN-N-0237:~$ kubectl apply -f nginx-service.yaml
service/nginx-service created
v_tu@PN-N-0237:~$ kubectl get pod

```

NAME	READY	STATUS	RESTARTS	AGE
nginx-depl-68c944fcbc-crxhv	1/1	Running	0	115m
nginx-deployment-7f65fcf556-7s62t	1/1	Running	0	88m
nginx-deployment-7f65fcf556-dz7dv	1/1	Running	0	86m

```

v_tu@PN-N-0237:~$ kubectl get service

```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	150m
nginx-service	ClusterIP	10.105.94.207	<none>	80/TCP	17s

```

v_tu@PN-N-0237:~$ kubectl describe service nginx-service
Name: nginx-service
Namespace: default
Labels: <none>
Annotations: <none>
Selector: app=nginx
Type: ClusterIP
IP Family Policy: SingleStack
IP Families: IPv4
IP: 10.105.94.207
IPs: 10.105.94.207
Port: <unset> 80/TCP
TargetPort: 8080/TCP
Endpoints: 10.244.0.5:8080,10.244.0.6:8080
Session Affinity: None
Internal Traffic Policy: Cluster
Events: <none>

```

```

v_tu@PN-N-0237:~$ kubectl get pod -o wide

```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED	NODE	READINESS GATES
nginx-depl-68c944fcbc-crxhv	1/1	Running	0	120m	10.244.0.3	minikube	<none>		<none>
nginx-deployment-7f65fcf556-7s62t	1/1	Running	0	93m	10.244.0.5	minikube	<none>		<none>
nginx-deployment-7f65fcf556-dz7dv	1/1	Running	0	91m	10.244.0.6	minikube	<none>		<none>

- kubectl get deployment nginx-deployment -o yaml

```

v_tu@PH-N-0237:~$ kubectl get deployment nginx-deployment -o yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  annotations:
    deployment.kubernetes.io/revision: "1"
    kubernetes.io/last-applied-configuration: |
      {"apiVersion":"apps/v1","kind":"Deployment","metadata":{"annotations":{},"labels":{"app":"nginx"},"name":"nginx-deployment","namespace":"default"},"spec":{"replicas":2,"selector":{"matchLabels":{"app":"nginx"}},"template":{"metadata":{"labels":{"app":"nginx"},"spec":{"containers":[{"image":"nginx:1.16","name":"nginx","ports":[{"containerPort":80}]}]}}}}
  creationTimestamp: "2025-05-28T09:48:27Z"
  generation: 2
  labels:
    app: nginx
  name: nginx-deployment
  namespace: default
  resourceVersion: "3495"
  uid: 87a0f97e-8c52-47b1-abee-24f48988cd13
spec:
  progressDeadlineSeconds: 600
  replicas: 2
  revisionHistoryLimit: 10
  selector:
    matchLabels:
      app: nginx
  strategy:
    rollingUpdate:
      maxSurge: 25%
      maxUnavailable: 25%
    type: RollingUpdate
  template:
    metadata:
      creationTimestamp: null
      labels:
        app: nginx
    spec:
      containers:
      - image: nginx:1.16
        imagePullPolicy: IfNotPresent
        name: nginx
        ports:
        - containerPort: 80
          protocol: TCP
        resources: {}
        terminationMessagePath: /dev/termination-log
        terminationMessagePolicy: File
      dnsPolicy: ClusterFirst
      restartPolicy: Always
      schedulerName: default-scheduler
      securityContext: {}
      terminationGracePeriodSeconds: 30
status:
  availableReplicas: 2
  conditions:
  - lastTransitionTime: "2025-05-28T09:48:27Z"
    lastUpdateTime: "2025-05-28T09:48:43Z"
    message: ReplicaSet "nginx-deployment-7f65fcf556" has successfully progressed.
    reason: NewReplicaSetAvailable
    status: "True"
    type: Progressing
  - lastTransitionTime: "2025-05-28T09:50:53Z"
    lastUpdateTime: "2025-05-28T09:50:53Z"
    message: Deployment has minimum availability.
    reason: MinimumReplicasAvailable
    status: "True"
    type: Available
  observedGeneration: 2
  readyReplicas: 2
  replicas: 2
  updatedReplicas: 2
v_tu@PH-N-0237:~$

```

- - Check containers configuration on <https://hub.docker.com/>

- https://hub.docker.com/_/mongo

How to use this image

Start a `mongo` server instance

```
$ docker run --name some-mongo -d mongo:tag
```

... where `some-mongo` is the name you want to assign to your container and `tag` is the tag specifying the MongoDB version you want. See the list above for relevant tags.

Connect to MongoDB from another Docker container

The MongoDB server in the image listens on the standard MongoDB `port`, `27017`, so connecting via Docker networks will be the same as connecting to a remote `mongod`. The following example starts another MongoDB container instance and runs the `mongosh` (use `mongo` with `4.x` versions) command line client against the original MongoDB container from the example above, allowing you to execute MongoDB statements against your database instance:

```
$ docker run -it --network some-network --rm mongo mongosh --host some-mongo test
```

... where `some-mongo` is the name of your original `mongo` container.

... via [docker compose](#) 

- Example `compose.yaml` for `mongo` :

Environment Variables

When you start the `mongo` image, you can adjust the initialization of the MongoDB instance by passing one or more environment variables on the `docker run` command line. Do note that none of the variables below will have any effect if you start the container with a data directory that already contains a database: any pre-existing database will always be left untouched on container startup.

MONGO_INITDB_ROOT_USERNAME, MONGO_INITDB_ROOT_PASSWORD

These variables, used in conjunction, create a new user and set that user's password. This user is created in the `admin` authentication database and given the role of `root`, which is a "superuser" role.

The following is an example of using these two variables to create a MongoDB instance and then using the `mongo` cli to connect against the `admin` authentication database.

```
$ docker run -d --network some-network --name some-mongo \
```

- New appli with secret

```
apiVersion: v1
kind: Secret
metadata:
  name: mongodb-secret
type: Opaque
data:
  mongo-root-username: dXNlcm5hbWU=
  mongo-root-password: cGFzc3dvcmQ=
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mongodb-deployment
  labels:
    app: mongodb
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mongodb
  template:
    metadata:
      labels:
        app: mongodb
    spec:
      containers:
        - name: mongodb
          image: mongo
          ports:
            - containerPort: 27017
          env:
            - name: MONGO_INITDB_ROOT_USERNAME
              valueFrom:
                secretKeyRef:
                  name: mongo-secret
                  key: mongo-root-username
            - name: MONGO_INITDB_ROOT_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: mongo-secret
                  key: mongo-root-password
```

○

~

```

v_tu@PN-N-0237:~$ kubectl apply -f mongo-secret.yaml
secret/mongodb-secret created
v_tu@PN-N-0237:~$ kubectl get secret
NAME          TYPE      DATA   AGE
mongodb-secret Opaque    2       16s
v_tu@PN-N-0237:~$ vim mongo-deployment.yaml
v_tu@PN-N-0237:~$ vim mongo-deployment.yaml
v_tu@PN-N-0237:~$ kubectl apply -f mongo-deployment.yaml
deployment.apps/mongodb-deployment created
v_tu@PN-N-0237:~$ kubectl get all
NAME                                     READY   STATUS                                RESTARTS   AGE
pod/mongodb-deployment-7c8d894848-9zhpp 0/1     CreateContainerConfigError           0          12s

NAME          TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)    AGE
service/kubernetes ClusterIP   10.96.0.1    <none>        443/TCP    5h46m

NAME                                     READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/mongodb-deployment 0/1     1             0          12s

NAME                                     DESIRED   CURRENT   READY   AGE
replicaset.apps/mongodb-deployment-7c8d894848 1         1         0       12s
v_tu@PN-N-0237:~$

```

-
- Service

```

---
apiVersion: v1
kind: Service
metadata:
  name: mongo-service
spec:
  selector:
    app: mongodb
  ports:
    - protocol: TCP
      port: 27017
      targetPort: 27017
---
apiVersion: v1
kind: Deployment
metadata:
  name: mongo-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mongodb
  template:
    metadata:
      labels:
        app: mongodb
    spec:
      containers:
        - name: mongodb
          image: mongo
          ports:
            - containerPort: 27017
          env:
            - name: MONGO_INITDB_ROOT_USERNAME
              valueFrom:
                secretKeyRef:
                  name: mongodb-secret
                  key: mongo-root-username
            - name: MONGO_INITDB_ROOT_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: mongodb-secret
                  key: mongo-root-password

```

"mongo-deployment.yaml" 44L, 861B

```

v_tu@PN-N-0237:~$ vim mongo-deployment.yaml
v_tu@PN-N-0237:~$ kubectl apply -f mongo-deployment.yaml
*deployment.apps/mongodb-deployment unchanged
service/mongo-service created
v_tu@PN-N-0237:~$ vim mongo-deployment.yaml
v_tu@PN-N-0237:~$ kubectl get service

```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	5d4h
mongo-service	ClusterIP	10.103.146.8	<none>	27017/TCP	105s

```

v_tu@PN-N-0237:~$

```

```

v_tu@PN-N-0237:~$ kubectl get service

```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	5d4h
mongo-service	ClusterIP	10.103.146.8	<none>	27017/TCP	105s

```

v_tu@PN-N-0237:~$ kubectl describe service mongo-service

```

```

Name: mongo-service
Namespace: default
Labels: <none>
Annotations: <none>
Selector: app=mongodb
Type: ClusterIP
IP Family Policy: SingleStack
IP Families: IPv4
IP: 10.103.146.8
IPs: 10.103.146.8
Port: <unset> 27017/TCP
TargetPort: 27017/TCP
Endpoints: 10.244.0.8:27017
Session Affinity: None
Internal Traffic Policy: Cluster
Events: <none>

```

```

v_tu@PN-N-0237:~$ kubectl get pod -o wide

```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED	NODE	READINESS	GATES
mongodb-deployment-6d9d7c68f6-grfdv	1/1	Running	0	4d22h	10.244.0.8	minikube	<none>		<none>	

```

v_tu@PN-N-0237:~$

```

```
v_tu@PN-N-0237:~$ kubectl get all | grep mongodb
pod/mongodb-deployment-6d9d7c68f6-grfdv 1/1 Running 0 4d22h
deployment.apps/mongodb-deployment 1/1 1 1 4d23h
replicaset.apps/mongodb-deployment-6d9d7c68f6 1 1 1 4d22h
replicaset.apps/mongodb-deployment-7c8d894848 0 0 0 4d23h
v_tu@PN-N-0237:~$
```

-
- MongoDB-express / configMap
 - database_url refers to the service:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: mongodb-configmap
data:
  database_url: mongodb-service
```

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: mongo-express
  labels:
    app: mongo-express
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mongo-express
  template:
    metadata:
      labels:
        app: mongo-express
    spec:
      containers:
        - name: mongo-express
          image: mongo-express
          ports:
            - containerPort: 8081
          env:
            - name: ME_CONFIG_MONGODB_AUTH_USERNAME
              valueFrom:
                secretKeyRef:
                  name: mongodb-secret
                  key: mongo-root-username
            - name: ME_CONFIG_MONGODB_AUTH_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: mongodb-secret
                  key: mongo-root-username
            - name: ME_CONFIG_MONGODB_AUTH_DATABASE
              valueFrom:
                configMapKeyRef:
                  name: mongodb-configmap
                  key: database_url

```

-
- kubectl apply -f mongodb-configmap.yaml
- kubectl apply -f mongo-express.yaml
- external service

```

      configMapKeyRef:
        name: mongodb-configmap
        key: database_url
---

```

```

apiVersion: v1
kind: Service
metadata:
  name: mongo-express-service
spec:
  selector:
    app: mongo-express
  type: LoadBalancer
  ports:
    - protocol: TCP
      port: 8081
      targetPort: 8081
      nodePort: 30000

```

- "mongo-express.yaml" 51L, 1118B

```

v_tu@PN-N-0237:~$ vim mongo-express.yaml
v_tu@PN-N-0237:~$ kubectl apply -f mongo-express.yaml
deployment.apps/mongo-express unchanged
service/mongo-express-service created
v_tu@PN-N-0237:~$ kubectl get pod

```

```

NAME                                READY   STATUS    RESTARTS   AGE
mongo-express-78fc69476c-76sgw      1/1     Running   4 (85s ago) 8m24s
mongodb-deployment-6d9d7c68f6-grfdv 1/1     Running   0           6d16h

```

```

v_tu@PN-N-0237:~$ kubectl get service

```

```

NAME                TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
kubernetes          ClusterIP   10.96.0.1       <none>        443/TCP          6d22h
mongo-express-service LoadBalancer 10.99.163.203   <pending>     8081:30000/TCP   15s
mongo-service       ClusterIP   10.103.146.8    <none>        27017/TCP        42h

```

- v_tu@PN-N-0237:~\$

```

v_tu@PN-N-0237:~$ minikube service mongo-express-service

```

NAMESPACE	NAME	TARGET PORT	URL
default	mongo-express-service	8081	http://192.168.39.44:30000

```

🚀 Opening service default/mongo-express-service in default browser...

```

```

👉 http://192.168.39.44:30000
v_tu@PN-N-0237:~$

```

- Namespace
 - Virtual cluster inside the cluster

- Example of namespace
 - Resources grouped: a namespace for DB, a namespace for mentoring, etc.
 - Teams grouped: for example, several teams use the same naming, creating conflict
 - Resource sharing:
 - Environment: staging, development + common resources (like elastic stack for example)
 - Blue/Green deployment: blue deploy, green deploy + common resources
 - Limit resources and/or access (example rights control, resource limitation, etc)
- **Cannot be shared between namespace: configMap or secret**, even for a same resource, need to be duplicated for each resource needed
- **Can be shared between namespace: service**
- External service VS Ingress: to access to the application
 - External service
 - Simple to use
 - useful for quick tests,
 - Expose port, example: 192.60.11.5:4123
 - Not secure (http)
 - Ingress
 - Expected result
 - Looks like a proper address: my-app/com
 - Can be secured (https), default http
- Ingress
 - <https://dev-k8sref-io.web.app/docs/services/ingress-v1/#ingressspec>

```

v_tu@PN-N-0237:~$ minikube addons enable ingress
💡 ingress is an addon maintained by Kubernetes. For any concerns contact minikube on GitHub.
You can view the list of minikube maintainers at: https://github.com/kubernetes/minikube/blob/master/OWNERS
▪ Using image registry.k8s.io/ingress-nginx/controller:v1.12.2
▪ Using image registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.5.3
▪ Using image registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.5.3
🔗 Verifying ingress addon...
🌟 The 'ingress' addon is enabled
v_tu@PN-N-0237:~$ kubectl get ns
NAME                STATUS   AGE
default             Active   6d23h
ingress-nginx       Active   59s
kube-node-lease     Active   6d23h
kube-public         Active   6d23h
kube-system         Active   6d23h
v_tu@PN-N-0237:~$ kubectl create namespace kubernetes-dashboard
namespace/kubernetes-dashboard created
v_tu@PN-N-0237:~$ kubectl get ns
NAME                STATUS   AGE
default             Active   6d23h
ingress-nginx       Active   99s
kube-node-lease     Active   6d23h
kube-public         Active   6d23h
kube-system         Active   6d23h
kubernetes-dashboard Active   2s
v_tu@PN-N-0237:~$

```

```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: dashboard-ingress
  namespace: kubernetes-dashboard
spec:
  rules:
  - host: dashboard.com
    http:
      paths:
      - pathType: Prefix
        path: "/"
        backend:
          service:
            name: kubernetes-dashboard
            port:
              number: 80

```

```

~
~
~

```

```
v_tu@PN-N-0237:~$ kubectl apply -f dashboard-ingress.yaml
ingress.networking.k8s.io/dashboard-ingress created
```

- ```
v_tu@PN-N-0237:~$ kubectl get Ingress --namespace=kubernetes-dashboard
NAME CLASS HOSTS ADDRESS PORTS AGE
dashboard-ingress nginx dashboard.com 192.168.39.44 80 116s
v_tu@PN-N-0237:~$ ^C
v_tu@PN-N-0237:~$ kubectl get Ingress -n kubernetes-dashboard
NAME CLASS HOSTS ADDRESS PORTS AGE
dashboard-ingress nginx dashboard.com 192.168.39.44 80 2m51s
v_tu@PN-N-0237:~$
```

- 
- sudo vim /etc/hosts

```
This file was automatically generated by WSL. To stop automatic generation of this file, add the following entry to /etc/wsl.conf:
[network]
generateHosts = false
127.0.0.1 localhost
127.0.1.1 PN-N-0237.sesame.infotel.com PN-N-0237

The following lines are desirable for IPv6 capable hosts
::1 ip6-localhost ip6-loopback
fe00::0 ip6-localnet
ff00::0 ip6-mcastprefix
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
192.168.39.44 dashboard.com
~
```

- 
- Volume
  - Storage that doesn't depend on the pod lifecycle (like a pod for mysql for e.g.)
- StatefulSet
  - for stateful application
  - can replicate pod
  - can run multiple replicates
- Service type
  - ClusterIP
    - Default
  - Headless
    - For statefull application (like db)

- NodePort
  - Call the port directly
  - Not secure
- LoadBalancer
  - Accessible and more secure than nodeport

## Installer et configurer kubectl | Kubernetes

- Vous devez utiliser une version de kubectl qui diffère seulement d'une version mineure de la version de votre cluster. Par exemple, un client v1.2 doit fonctionner avec un master v1.1, v1.2 et v1.3. L'utilisation de la dernière version de kubectl permet d'éviter des problèmes imprévus.
- Installer **KVM** (hyperviseur) sur Ubuntu
  - <https://computingforgeeks.com/install-kvm-virtualization-on-ubuntu-noble-numbat/>
  - Configuration / Ajouter les permissions : <https://help.ubuntu.com/community/KVM/Installation>

```

v_tu@PN-N-0237:~$ groups
v_tu adm dialout cdrom floppy sudo audio dip video plugdev users netdev libvirt docker kvm
v_tu@PN-N-0237:~$ minikube start --driver=kvm
🐳 minikube v1.36.0 on Ubuntu 24.04 (amd64)
💡 Using the kvm2 driver based on user configuration
📦 Downloading driver docker-machine-driver-kvm2:
> docker-machine-driver-kvm2-...: 65 B / 65 B [-----] 100.00% ? p/s 0s
> docker-machine-driver-kvm2-...: 15.20 MiB / 15.20 MiB 100.00% 23.01 MiB
📀 Downloading VM boot image ...
> minikube-v1.36.0-amd64.iso....: 65 B / 65 B [-----] 100.00% ? p/s 0s
> minikube-v1.36.0-amd64.iso: 360.83 MiB / 360.83 MiB 100.00% 22.70 MiB p
👉 Starting "minikube" primary control-plane node in "minikube" cluster
📦 Downloading Kubernetes v1.33.1 preload ...
> preloaded-images-k8s-v18-v1...: 347.04 MiB / 347.04 MiB 100.00% 24.12 M
🔥 Creating kvm2 VM (CPUs=2, Memory=2200MB, Disk=20000MB) ...
❗ Image was not built for the current minikube version. To resolve this you can delete and recreate
your minikube cluster using the latest images. Expected minikube version: v1.35.0 -> Actual minikube v
ersion: v1.36.0
🔧 Preparing Kubernetes v1.33.1 on Docker 28.0.4 ...
 ▪ Generating certificates and keys ...
 ▪ Booting up control plane ...
 ▪ Configuring RBAC rules ...
🔗 Configuring bridge CNI (Container Networking Interface) ...
🔧 Verifying Kubernetes components...
 ▪ Using image gcr.io/k8s-minikube/storage-provisioner:v5
🌞 Enabled addons: storage-provisioner, default-storageclass
👉 Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
v_tu@PN-N-0237:~$ virsh list --all
 Id Name State

 1 minikube running

v_tu@PN-N-0237:~$ minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured

v_tu@PN-N-0237:~$

```